

Genetic Algorithm for Gas Turbine Health Indicator Estimation

Experiment Framework on a single measurment.

This notebook implements a **Genetic Algorithm (GA)** to solve the **inverse problem** in gas turbine engine health monitoring.

Systematic Experiments with Multiple Scenarios

This notebook includes:

1. **All original plots** (convergence, 2D/3D search space, ablation)
 2. **Multi-scenario experiments** (1-4 contexts × 1-3 sensors)
 3. **Clean and noisy data** comparison
 4. **Uses existing original and noisy CSV file**
-

For details about the problem and the parameter tuning please refer to :

- the `readme file`
- The `technical documentation` inside the Reports folder

Note : Update the `ga_config.py` if you have new configuration for the GA

```
In [9]: import sys
        from pathlib import Path

        # Add project root to sys.path
        project_root = Path().resolve().parent
        sys.path.append(str(project_root))
```

```
In [10]: project_root
```

```
Out[10]: PosixPath('/home/ensai/Documents/dataviz-course/SDSPROJECT/TO_PUSH/TO_PUSH_TRUE/scenarios/V3SAFRAN_ENSAI_2025/CLIMB2_genetic_algorithm/simulationv1')
```

```
In [11]: # include GA packages
        from functions.GA_single_experiment import *

        from functions.ga_config import *
```

```
In [ ]:
```

```
In [12]: %matplotlib inline
        import matplotlib.pyplot as plt
        from mpl_toolkits.mplot3d import Axes3D
        import numpy as np
        import pandas as pd
        from typing import List, Dict, Tuple, Optional
        from dataclasses import dataclass
        import time
        import os
        import pickle
        import warnings
        from datetime import datetime
        import copy
```

```
warnings.filterwarnings('ignore')

# Simulator imports
from odsmr.sensors import HPC_Tout, HP_Nmech, HPC_Tin, LPT_Tin, Fuel_flow, HPC_Pout_s
from odsmr.predefined_flight_conditions import Cruise_DeckSMR, Takeoff_DeckSMR, Climb
from odsmr.generation_functions import decksmr_lforall
from odsmr.constants import ROOT_OPENDECK, STATE_LABELS, STATE_BOUNDS

from scipy.stats import qmc

np.random.seed(42)
print("Imports OK")
```

Imports OK

```
In [13]: print("="*80)
print("ALL 10 HEALTH INDICATORS")
print("="*80)
print(f"{'Index':<6} {'Label':<35} {'Lower':>10} {'Upper':>10}")
print("-"*65)
for i, label in enumerate(STATE_LABELS):
    bounds = STATE_BOUNDS[label]
    print(f"{'i':<6} {'label':<35} {'bounds[0]:>10.3f} {'bounds[1]:>10.3f}")
```

=====

ALL 10 HEALTH INDICATORS

=====

Index	Label	Lower	Upper
0	deg_CmpBst_s_mapEff_in	-0.050	0.000
1	deg_CmpBst_s_mapWc_in	-0.050	0.030
2	deg_CmpFan_s_mapEff_in	-0.050	0.000
3	deg_CmpFan_s_mapWc_in	-0.050	0.030
4	deg_CmpH_s_mapEff_in	-0.050	0.000
5	deg_CmpH_s_mapWc_in	-0.050	0.030
6	deg_TrbH_s_mapEff_in	-0.050	0.000
7	deg_TrbH_s_mapWc_in	-0.050	0.050
8	deg_TrbL_s_mapEff_in	-0.050	0.000
9	deg_TrbL_s_mapWc_in	-0.050	0.050

1. Configuration of the simulator

Note: Update the `ga_config.py` if you have new configuration for the GA

```
In [14]: print(f"Contexts available: {ALL_CONTEXTS}")
print(f"Sensors available: {ALL_SENSORS}")
print(f"Indicators to estimate: {[INDICATOR_SHORT_NAMES[i] for i in INDICATORS_TO_EST"]
```

Contexts available: ['CLIMB2']
Sensors available: ['HPC_Tin', 'LPT_Tin', 'HPC_Pout_st']
Indicators to estimate: ['Fan Wc', 'HPC Eff', 'HPT Eff']

```
In [15]: print (SENSOR_SCENARIOS)

[['HPC_Tin'], ['HPC_Tin', 'LPT_Tin'], ['HPC_Tin', 'LPT_Tin', 'HPC_Pout_st']]
```

```
In [16]: print(f"Population: {config.population_size}, Elitism: {config.elitism_count}, Tourna
```

Population: 150, Elitism: 7, Tournament: 15

Load the data

```
In [17]: # Load clean data
df_clean = pd.read_csv(CLEAN_DATA_PATH)
print(f"Loaded {len(df_clean)} rows from {CLEAN_DATA_PATH}")
```

Loaded 1000 rows from ../data/synthetic_data.csv

```
In [ ]:
```

```
In [18]: # LOAD noisy version of data
df_noisy = pd.read_csv(NOISY_DATA_PATH)
print(f"Loaded {len(df_clean)} rows from {NOISY_DATA_PATH}")

# Show noise effect
print("\nNoise effect sample (row 0, CRUISE):")
for s in ALL_SENSORS:
    col = f'CRUISE_DECKSMR{s}'
    clean_val = df_clean.loc[0, col]
    noisy_val = df_noisy.loc[0, col]
    diff = noisy_val - clean_val
    print(f"  {s}: clean={clean_val:.4f}, noisy={noisy_val:.4f}, noise={diff:.4f}")
```

Loaded 1000 rows from ../data/synthetic_data_noisy.csv

Noise effect sample (row 0, CRUISE):

HPC_Tin: clean=336.0642, noisy=336.0353, noise=-0.0289

LPT_Tin: clean=1123.0515, noisy=1123.1817, noise=0.1302

HPC_Pout_st: clean=1377933.0900, noisy=1377498.8378, noise=-434.2522

```
In [ ]:
```

2. GA Configuration and Class

Note: Update the `ga_config.py` if you want new configuration for the GA

3. Run All Experiments

Single measurement estimation + Ablation study

```
In [19]: %%time
# Run all experiments
output_dir, summary_df = run_all_experiments(
    df_clean=df_clean,
    df_noisy=df_noisy,
    n_test_rows=N_TEST_ROWS,
    row_to_use=ROW_TO_USE,

    output_base_dir=OUTPUT_BASE_DIR
)
```

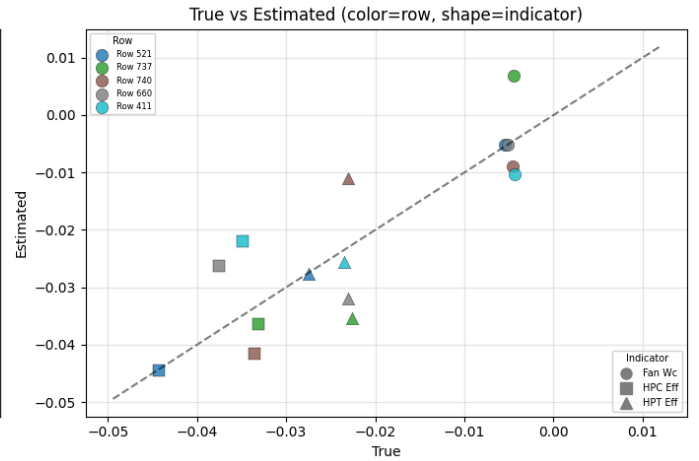
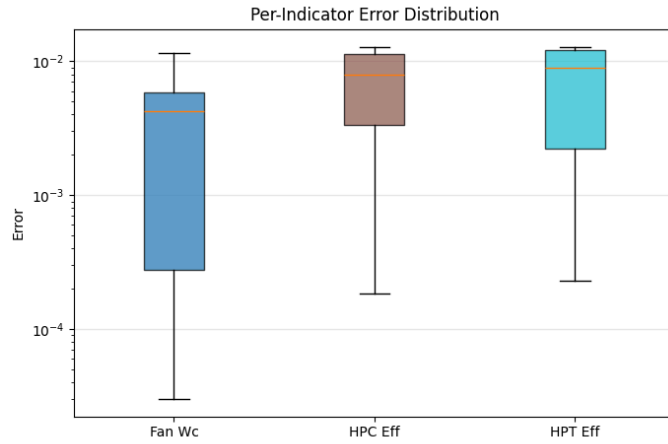
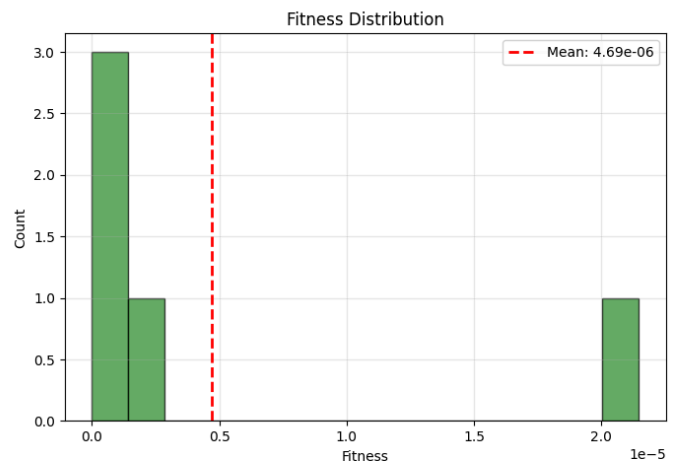
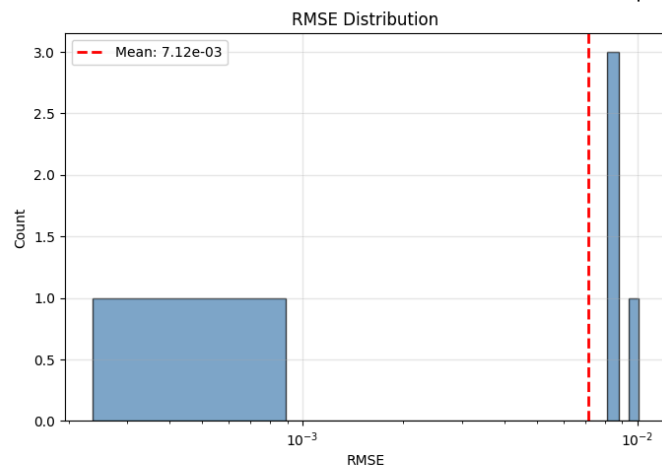
=====

Running experiments on CLEAN data

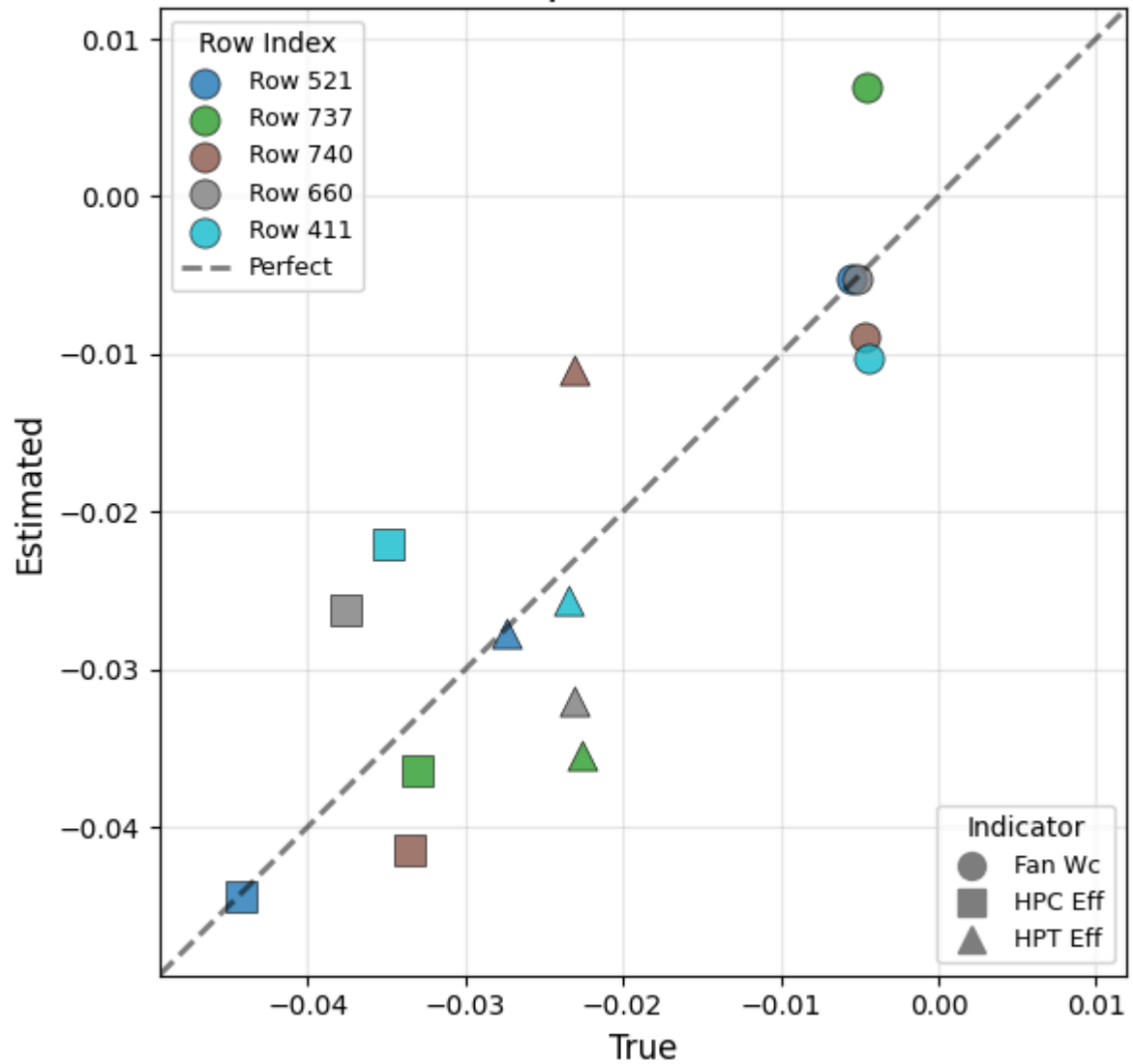
=====

lctx_1sensors: ['CLIMB2'] × ['HPC_Tin']
1 measurements → 3 indicators

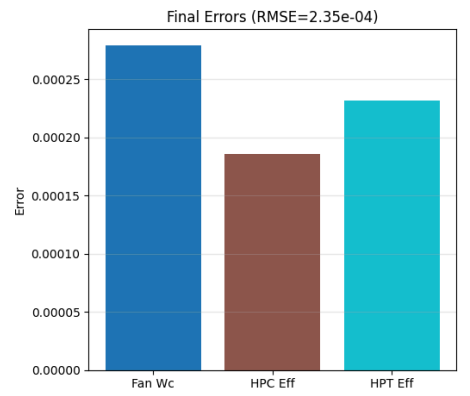
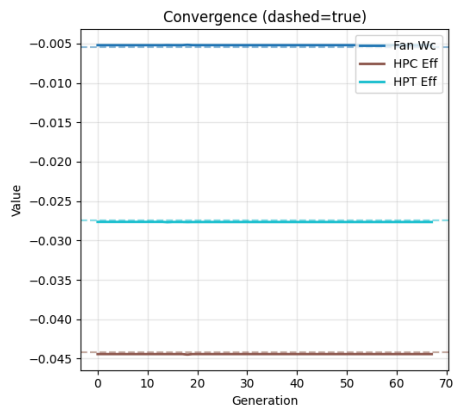
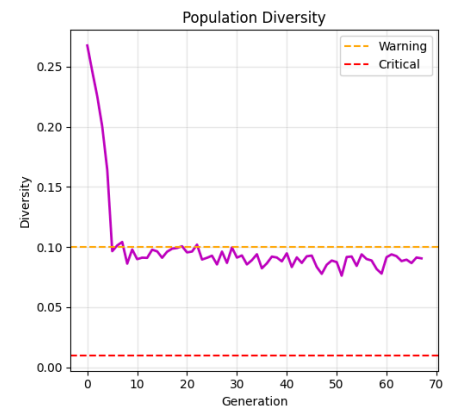
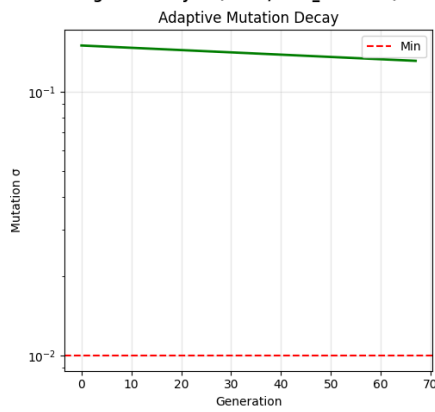
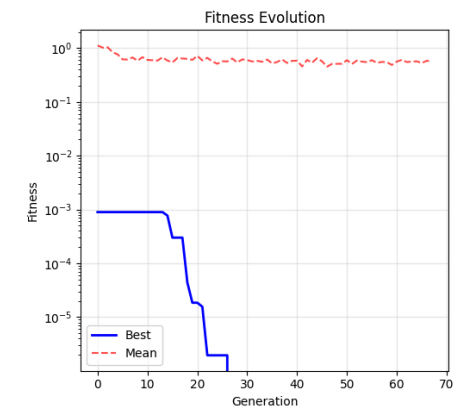
Clean | 1 ctx × 1 sens



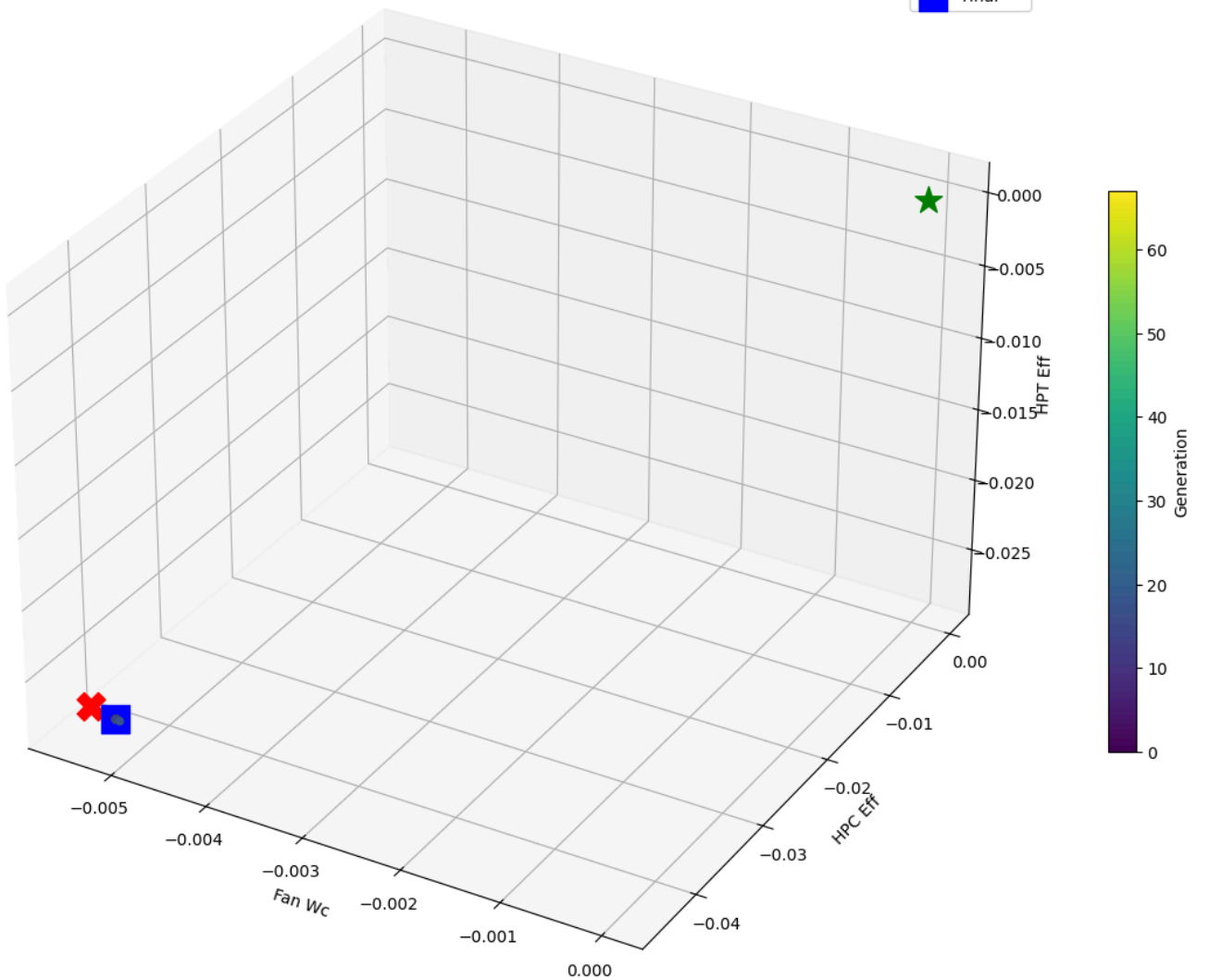
Clean | 1 ctx × 1 sens

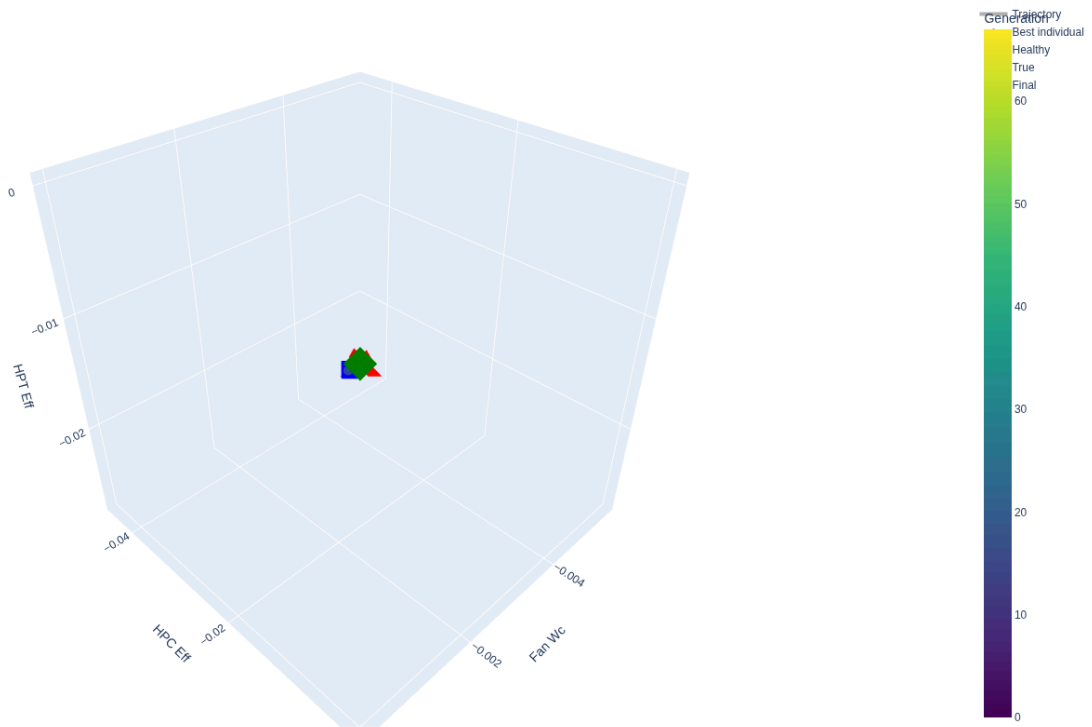


Convergence Analysis (clean, 1ctx_1sensors)



3D Search Trajectory (clean, 1ctx_1sensors)

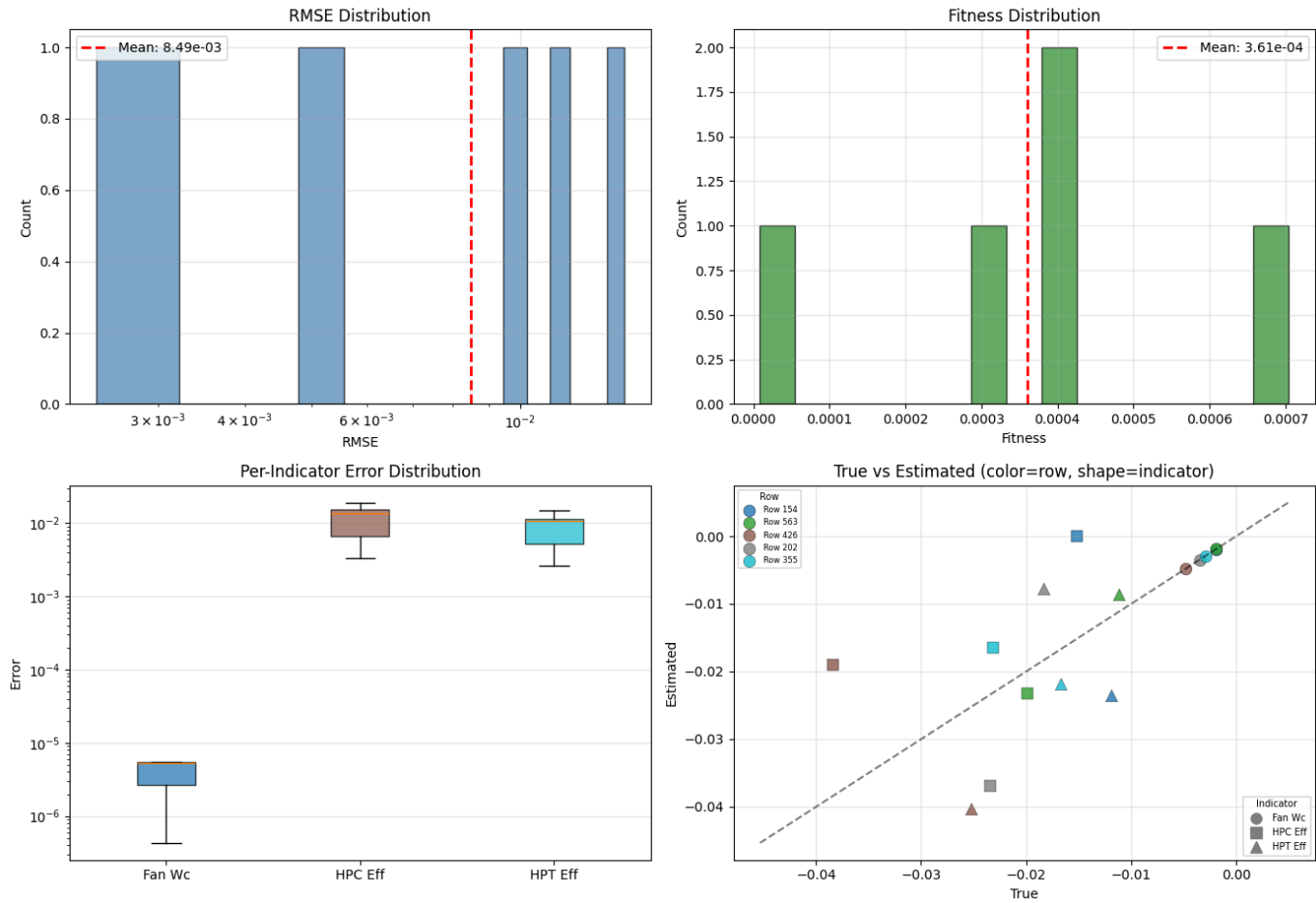




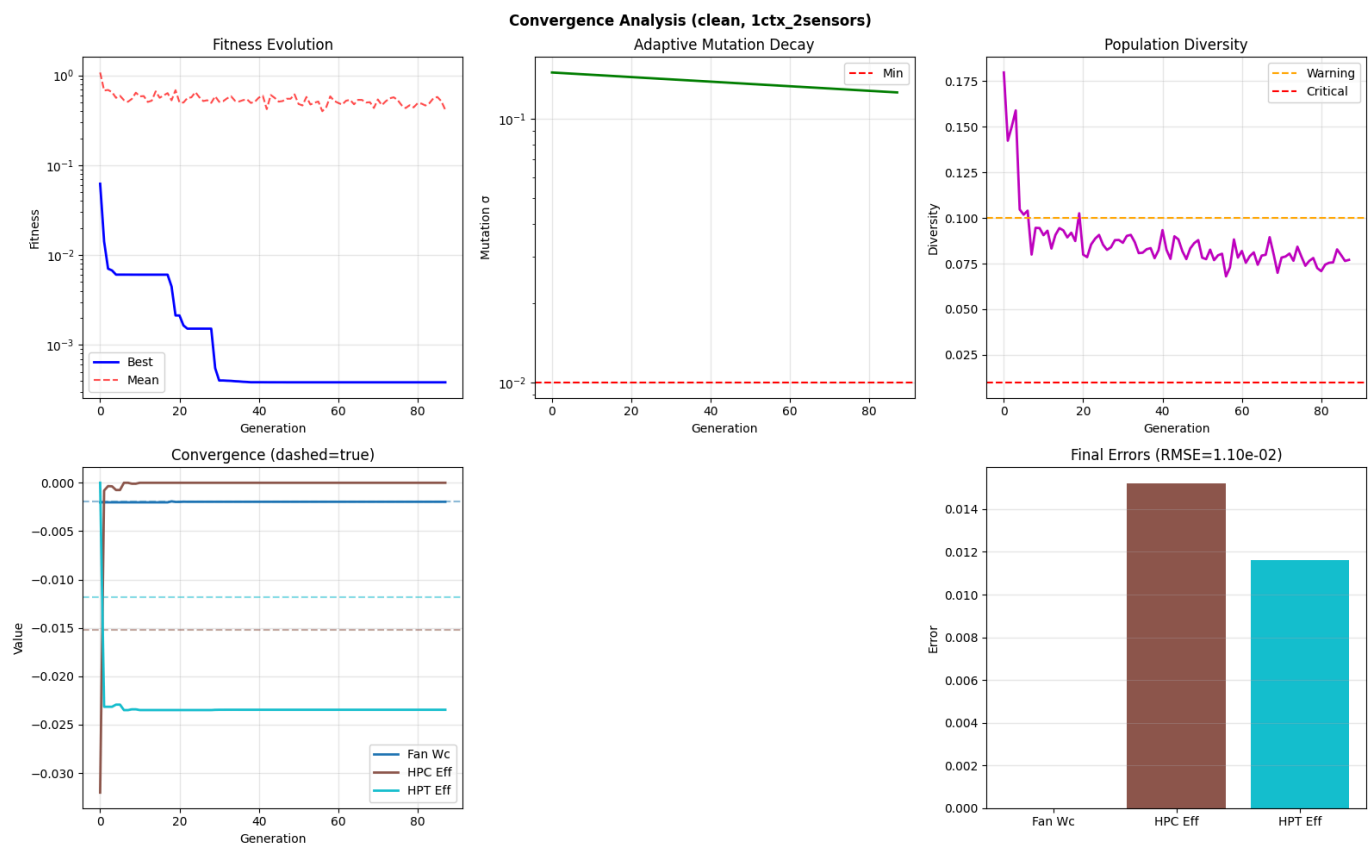
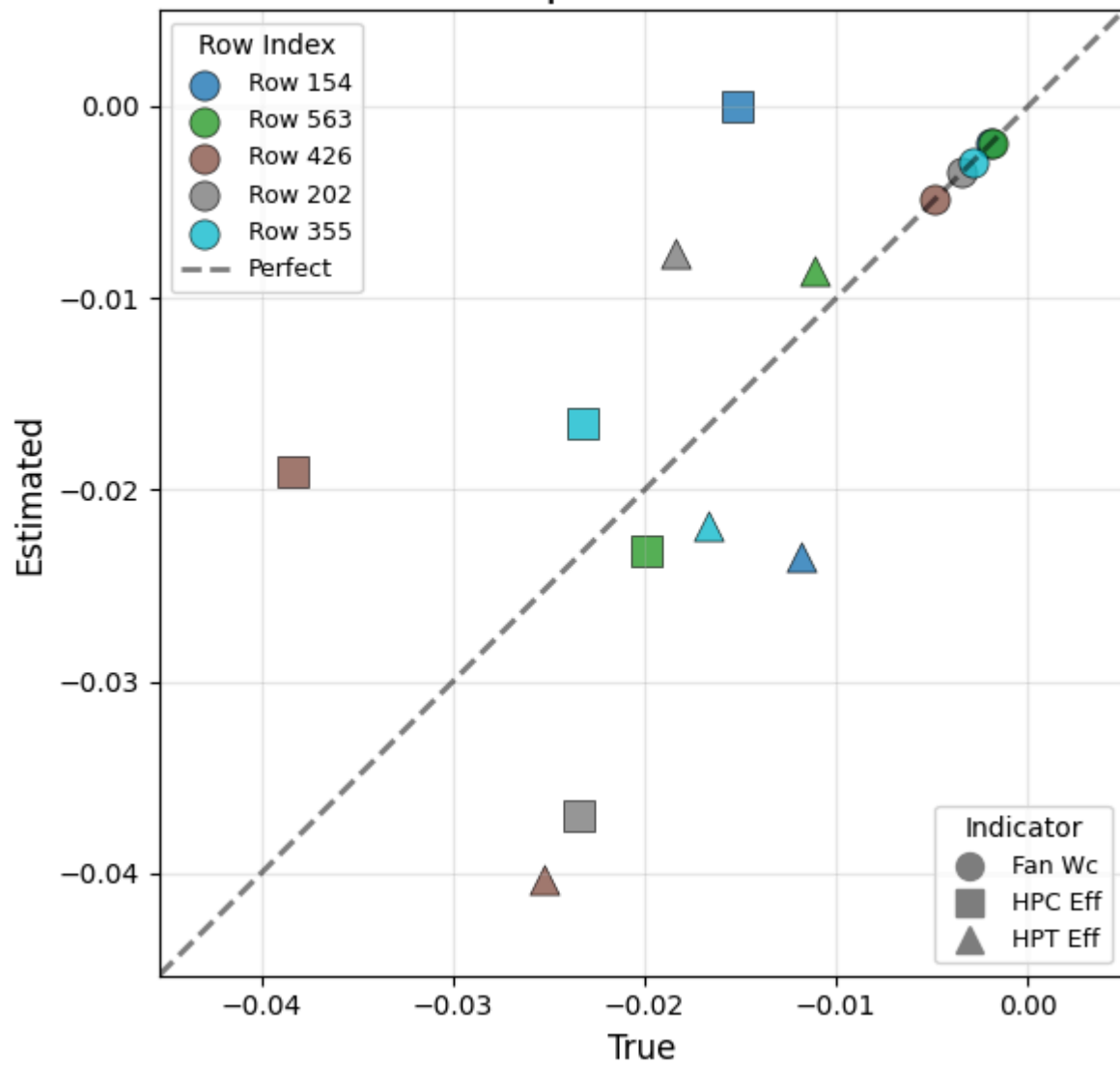
Mean RMSE: $7.12e-03 \pm 3.92e-03$
Mean Time: $1795.27s \pm 579.44s$

1ctx_2sensors: ['CLIMB2'] × ['HPC_Tin', 'LPT_Tin']
2 measurements → 3 indicators

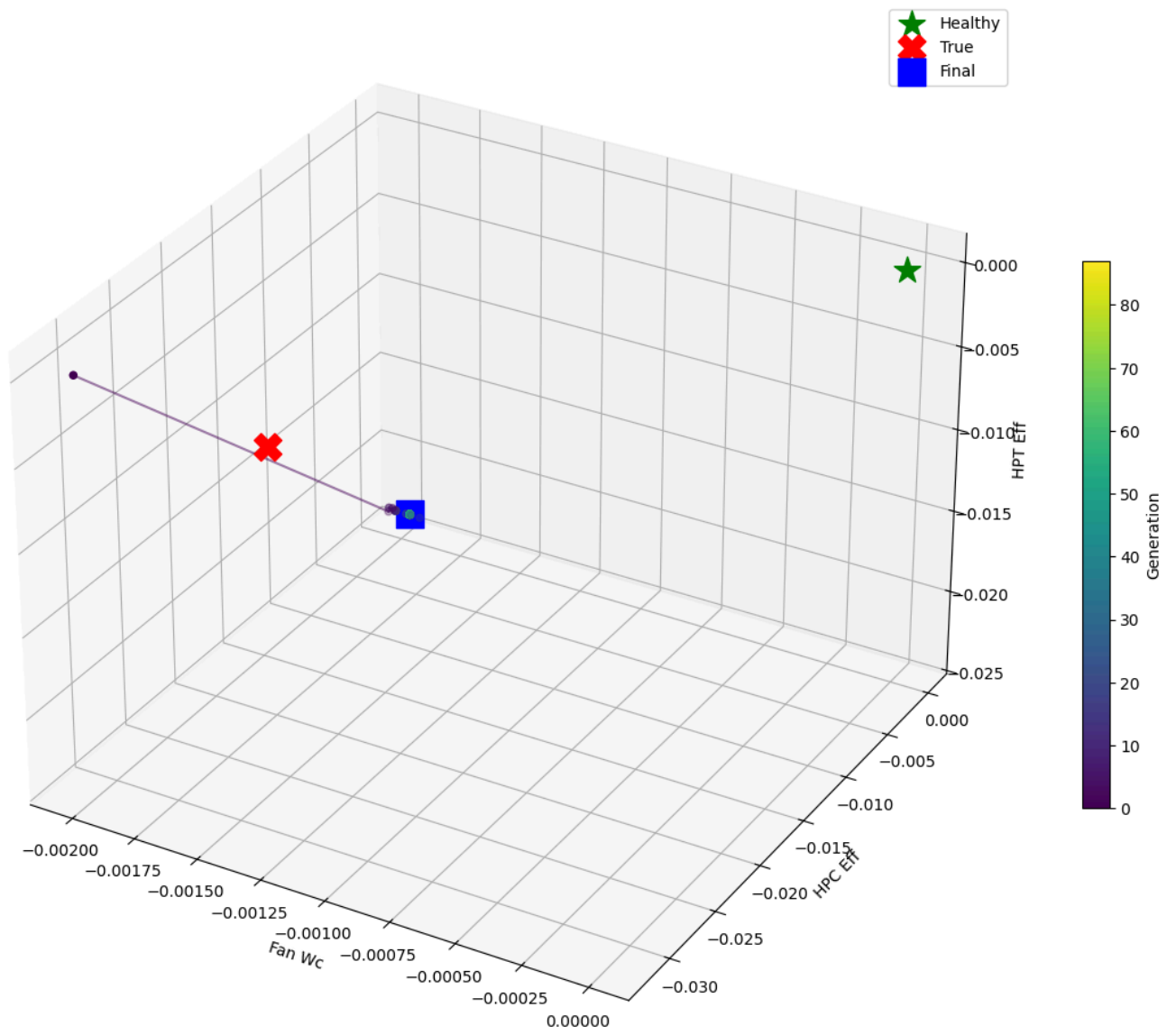
Clean | 1 ctx × 2 sens



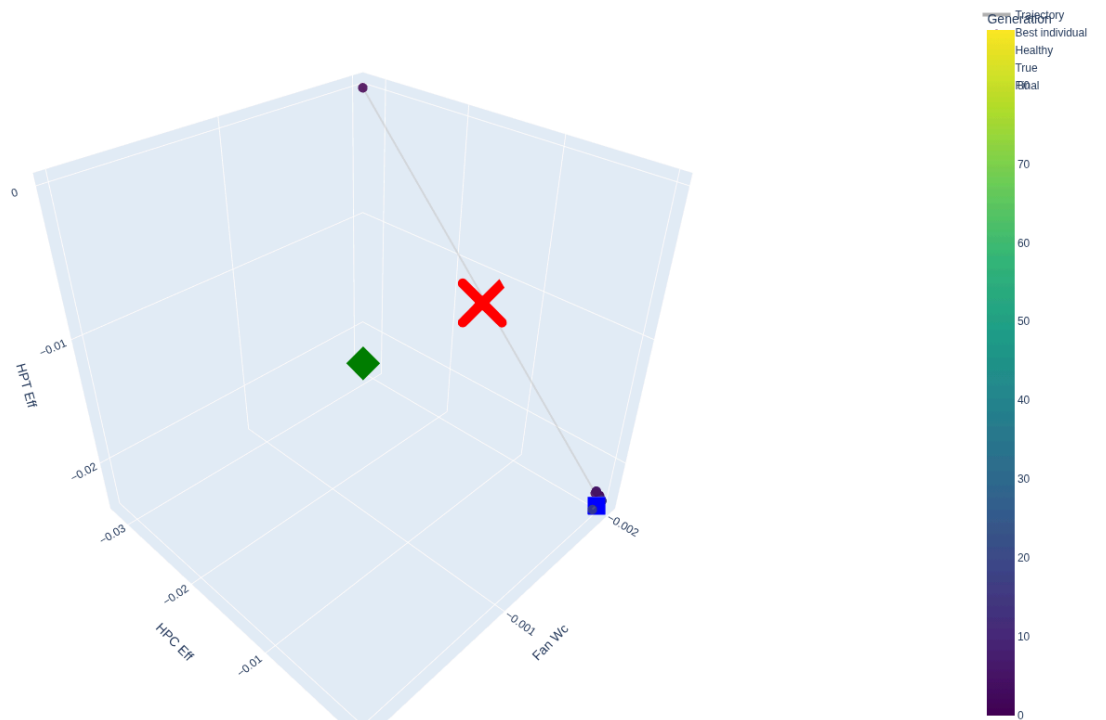
Clean | 1 ctx × 2 sens



3D Search Trajectory (clean, 1ctx_2sensors)

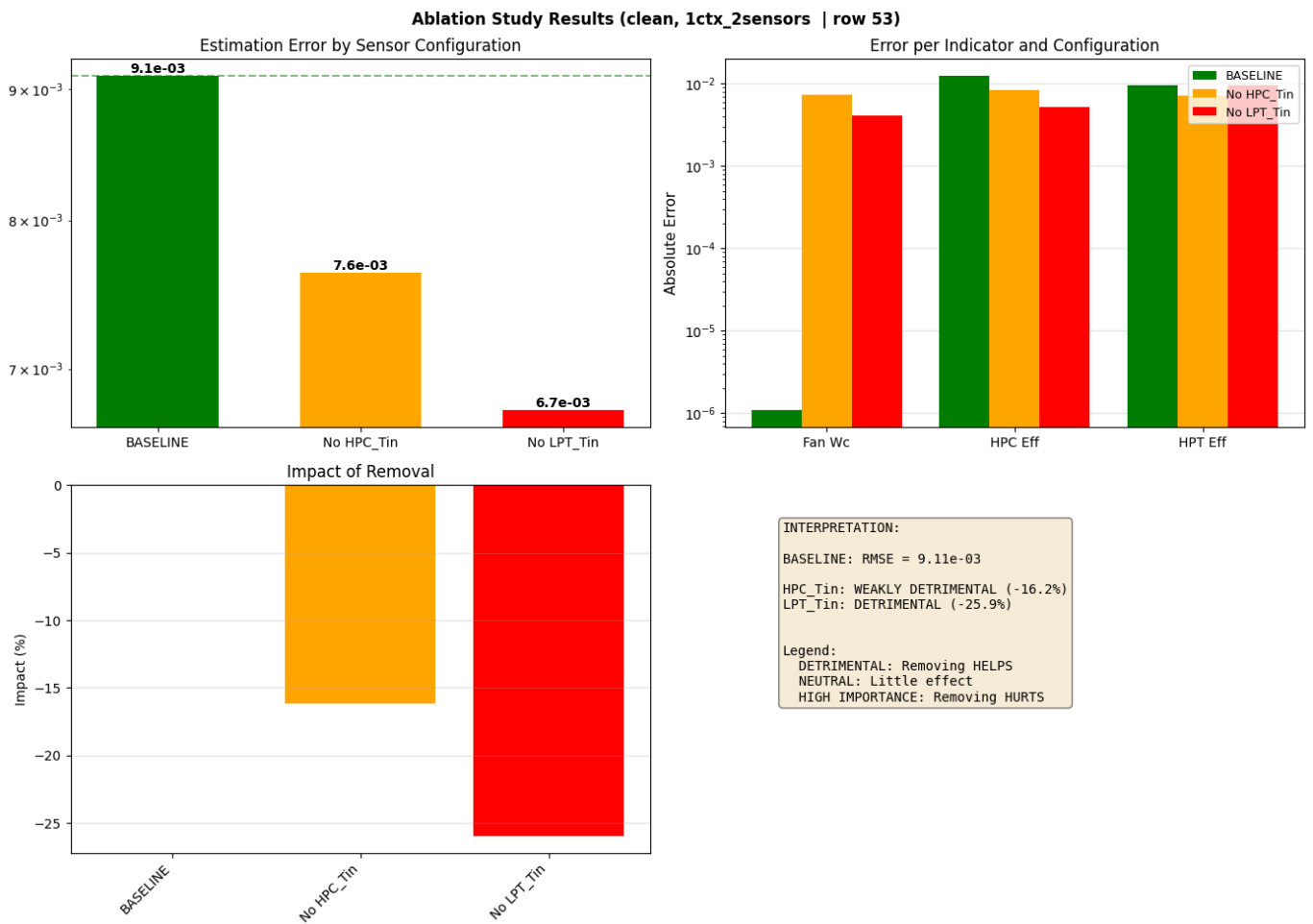
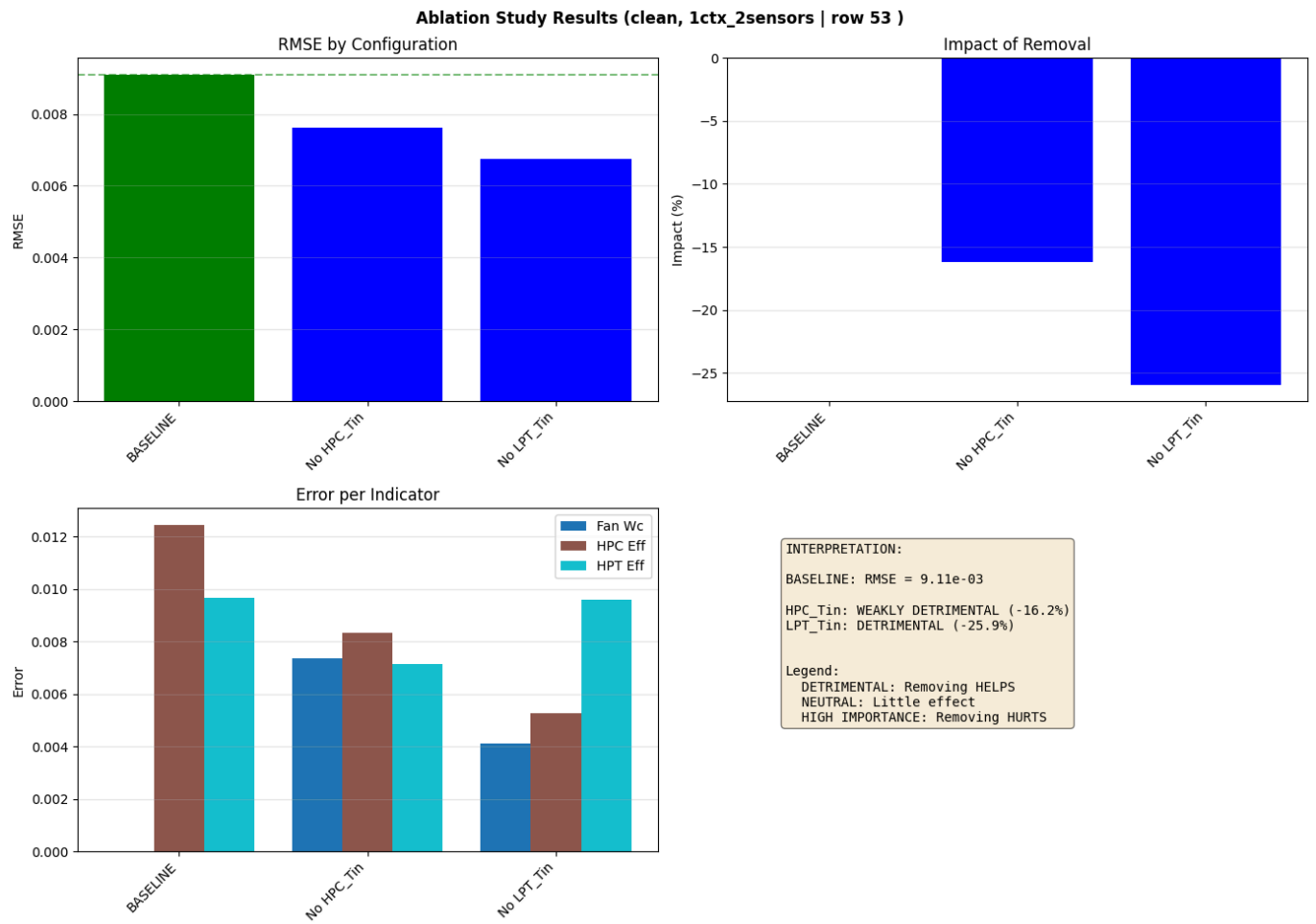


GA Search Trajectory



Running ablation study...
Running BASELINE...
RMSE: 9.11e-03

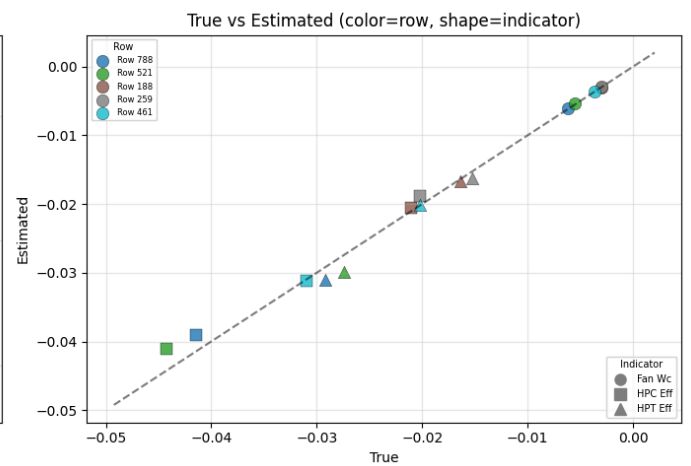
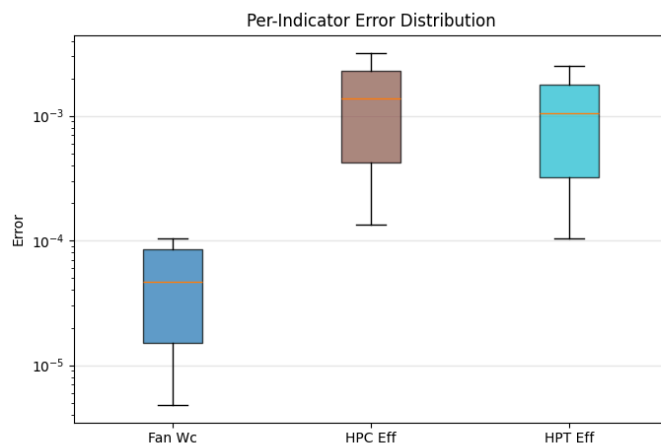
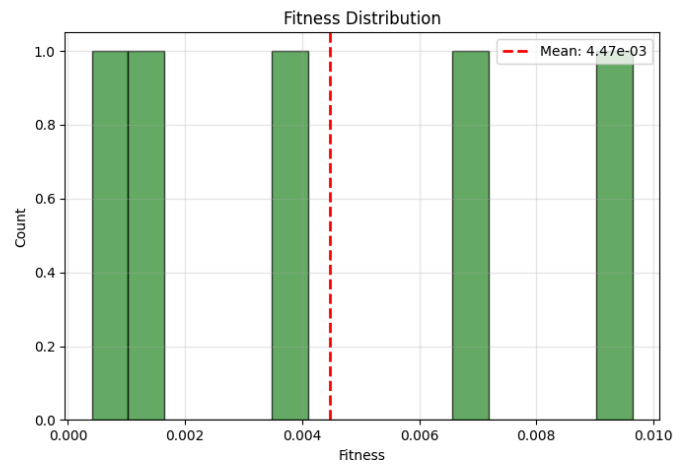
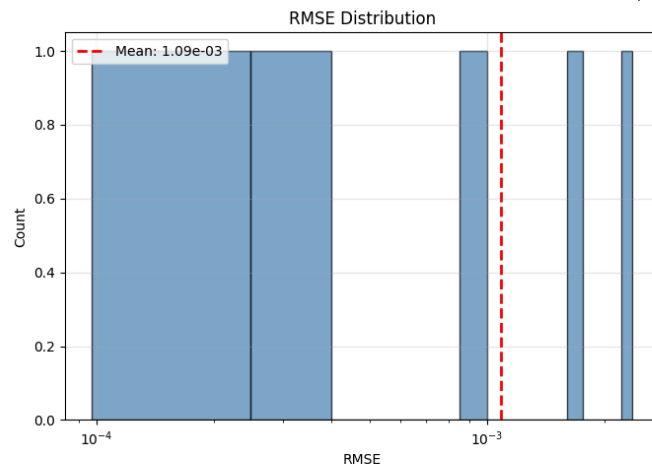
Sensor ablation:
Without HPC_Tin... RMSE: 7.63e-03 (-16.2%)
Without LPT_Tin... RMSE: 6.74e-03 (-25.9%)



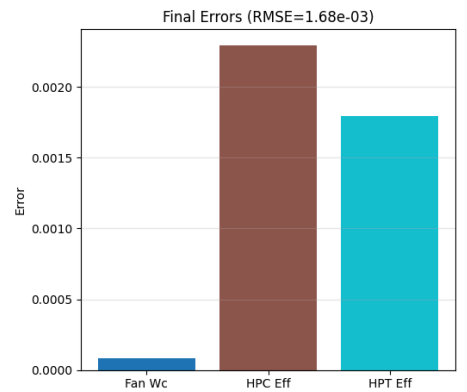
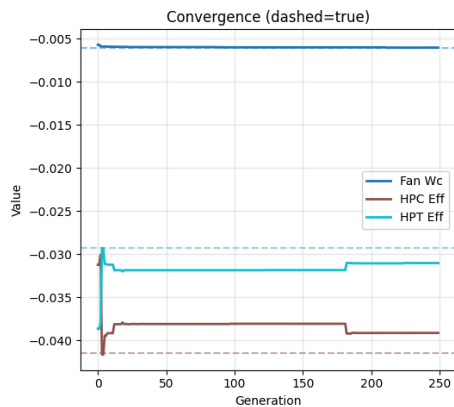
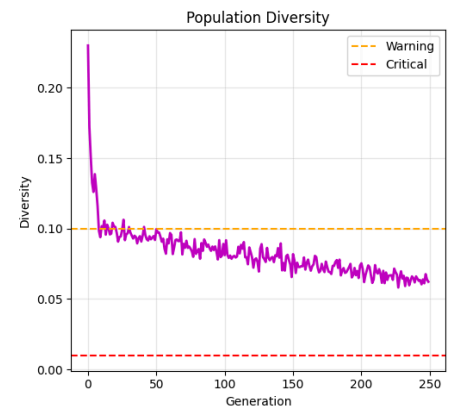
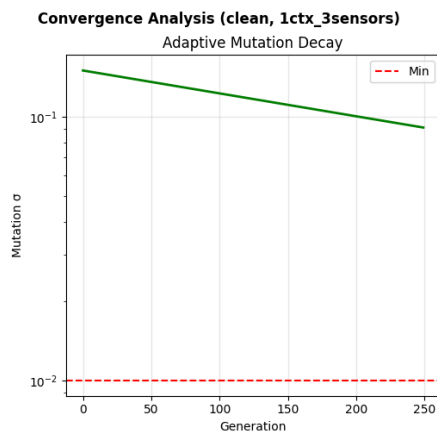
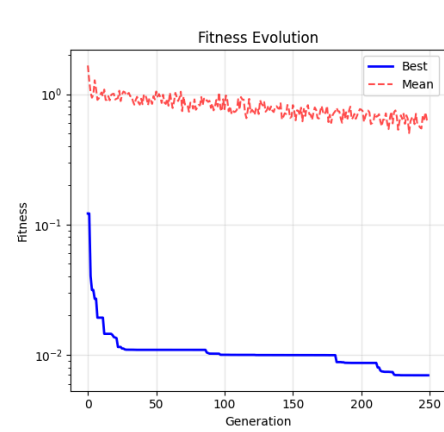
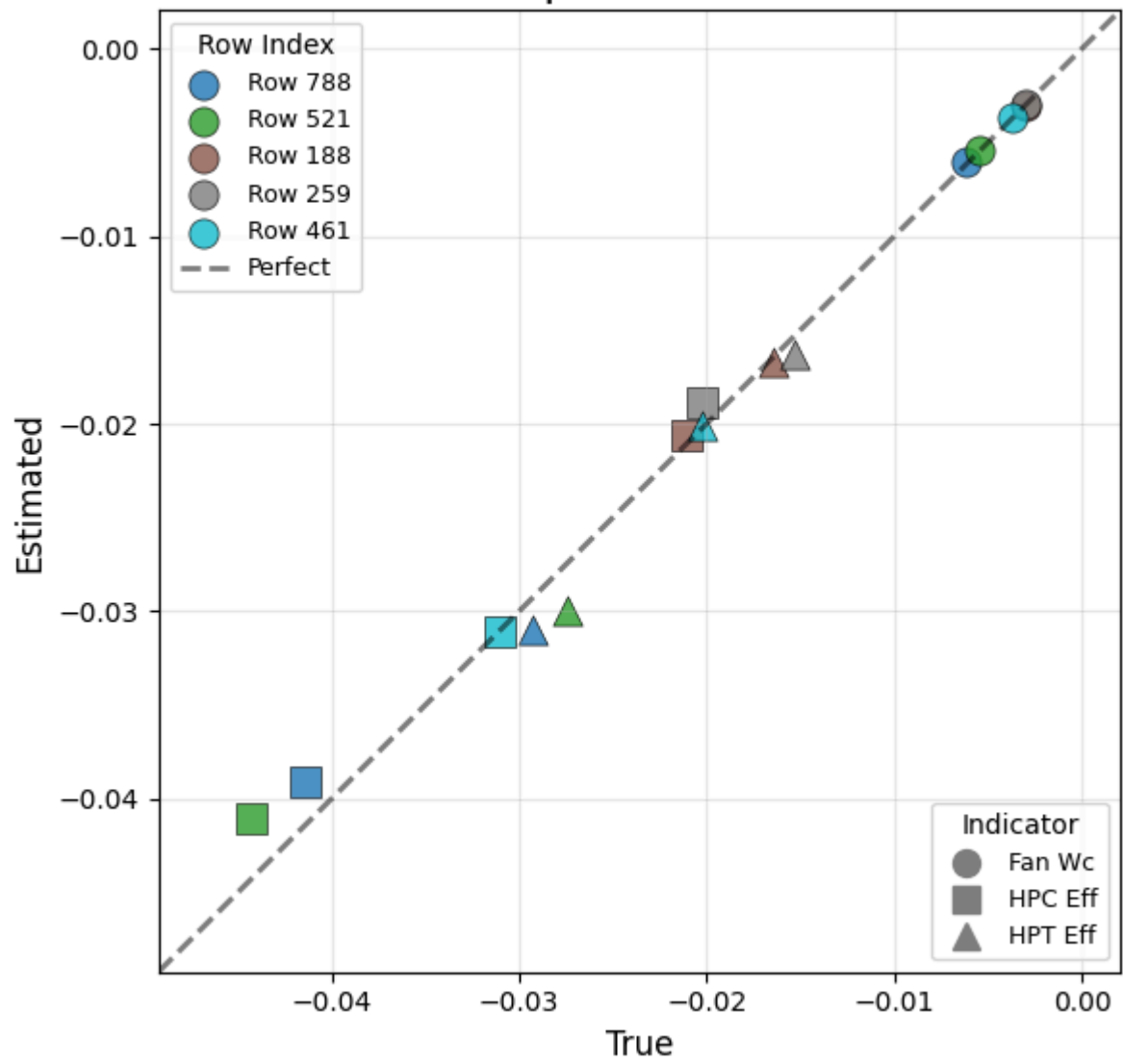
Mean RMSE: $8.49\text{e-}03 \pm 4.75\text{e-}03$
Mean Time: 2430.19s $\pm$ 560.75s

1ctx_3sensors: ['CLIMB2'] $\times$ ['HPC_Tin', 'LPT_Tin', 'HPC_Pout_st']
3 measurements $\rightarrow$ 3 indicators

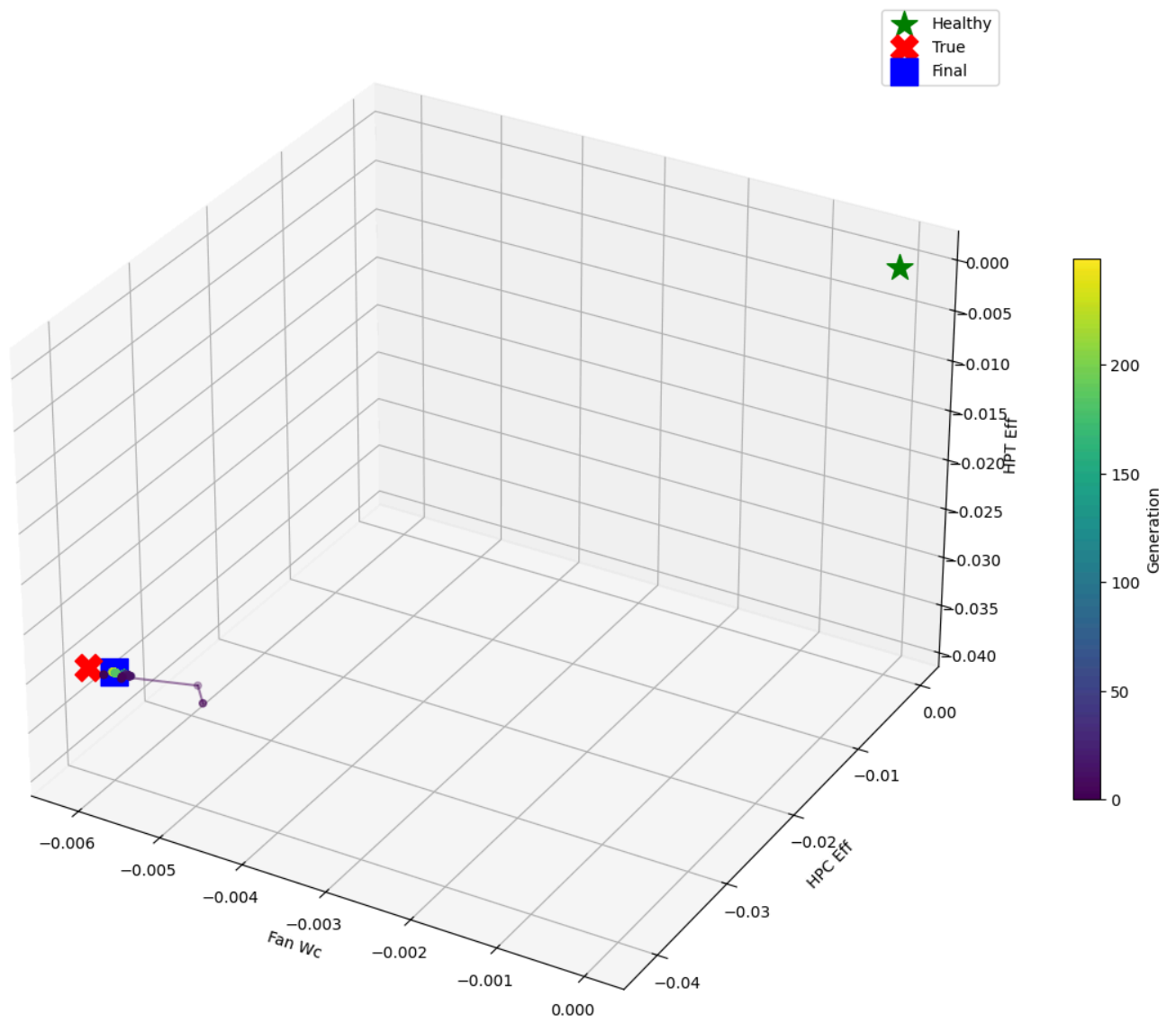
Clean | 1 ctx $\times$ 3 sens



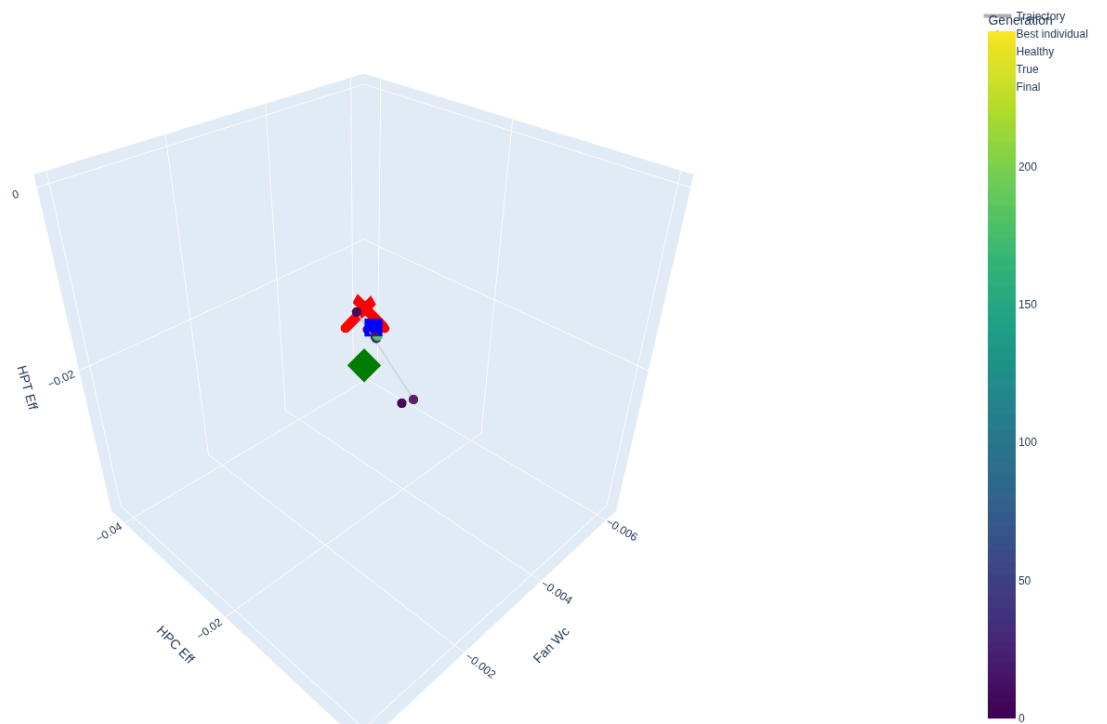
Clean | 1 ctx x 3 sens



3D Search Trajectory (clean, 1ctx_3sensors)

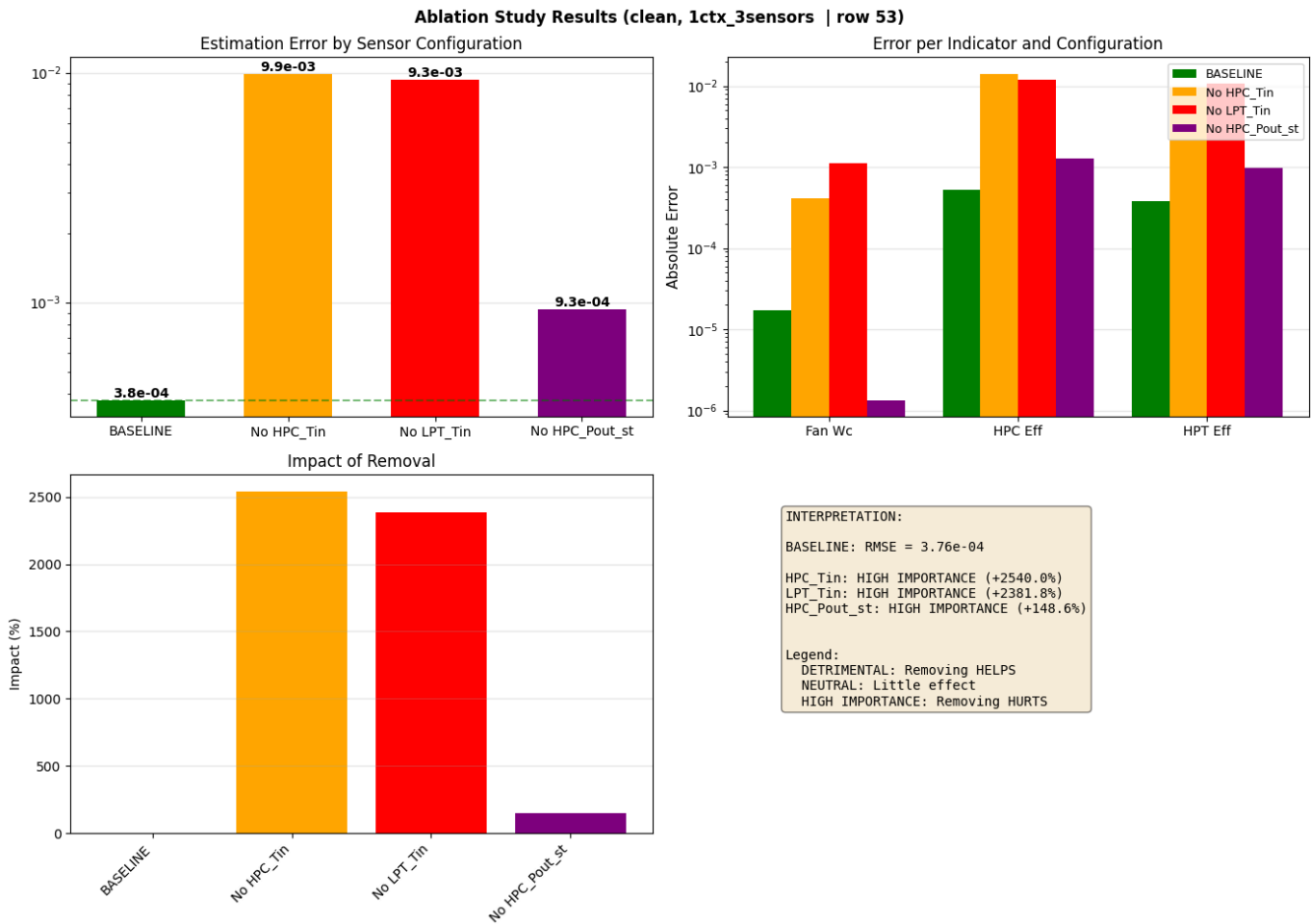
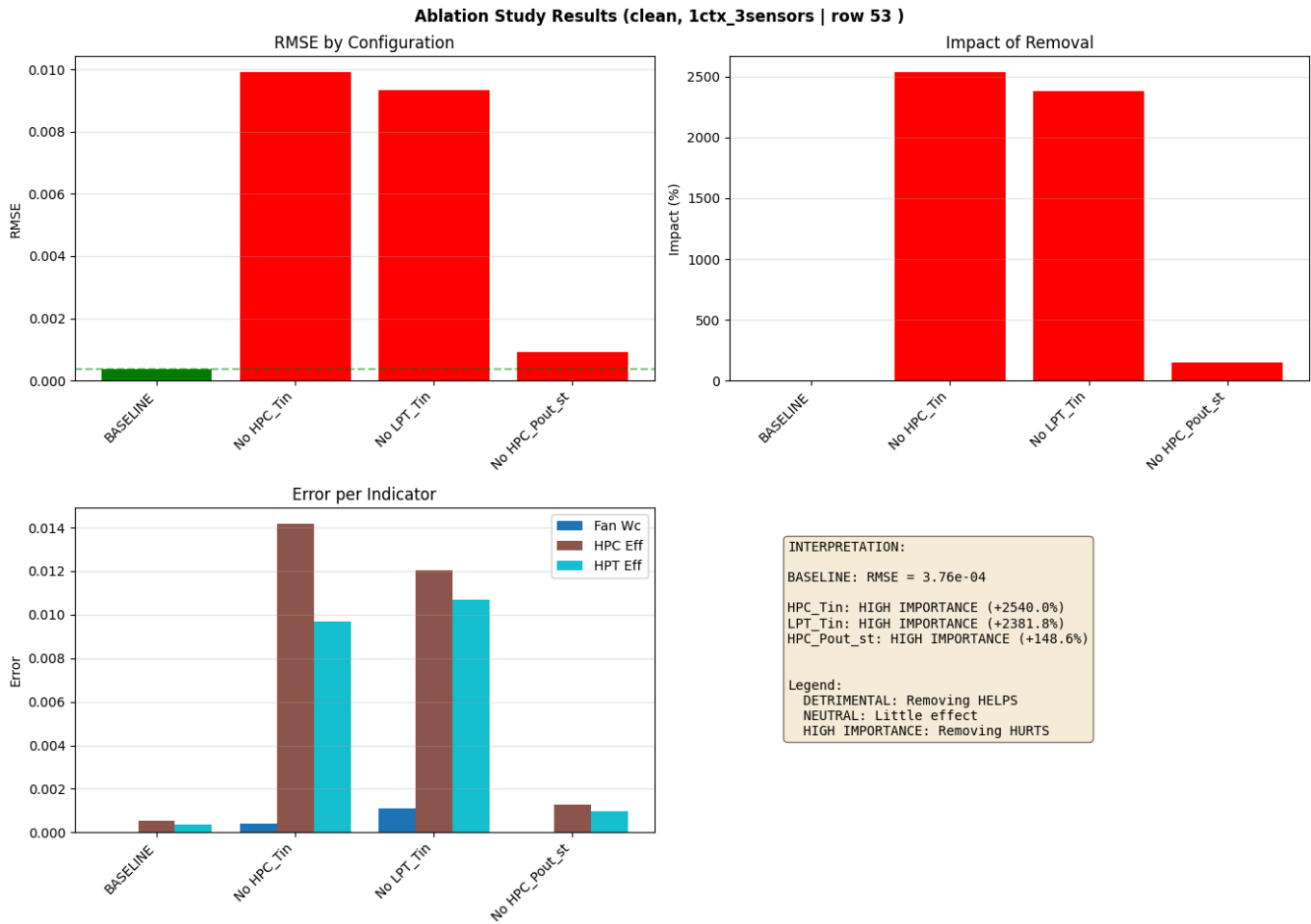


GA Search Trajectory



Running ablation study...
Running BASELINE...
RMSE: 3.76e-04

Sensor ablation:
Without HPC_Tin... RMSE: 9.92e-03 (+2540.0%)
Without LPT_Tin... RMSE: 9.33e-03 (+2381.8%)
Without HPC_Pout_st... RMSE: 9.34e-04 (+148.6%)

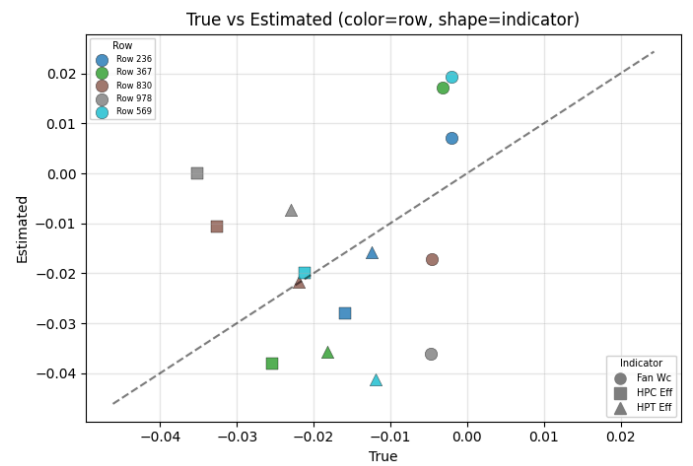
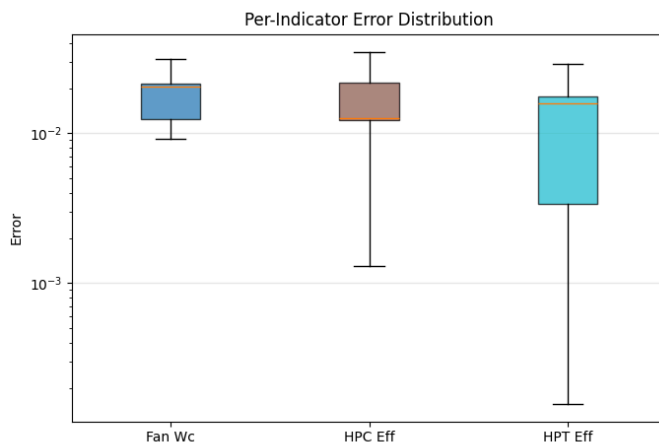
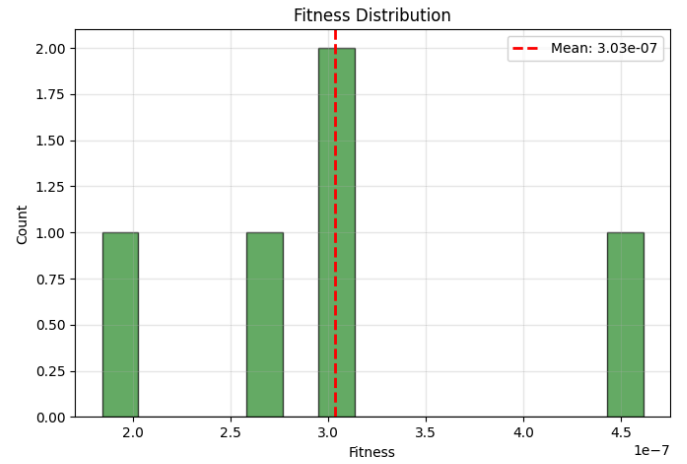
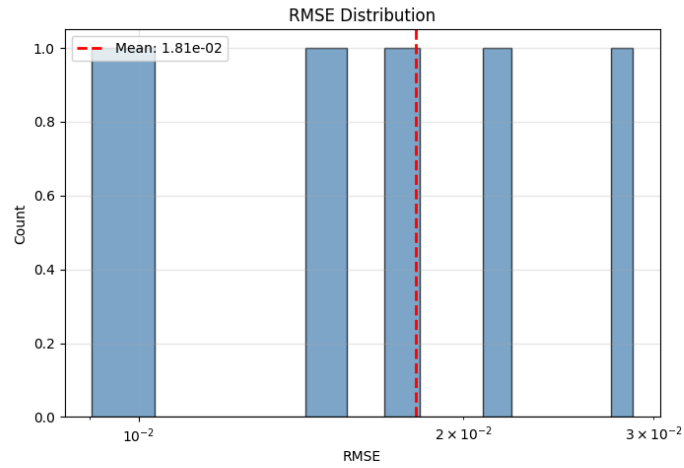


Mean RMSE: $1.09\text{e-}03 \pm 9.45\text{e-}04$
Mean Time: $5717.00\text{s} \pm 1050.81\text{s}$

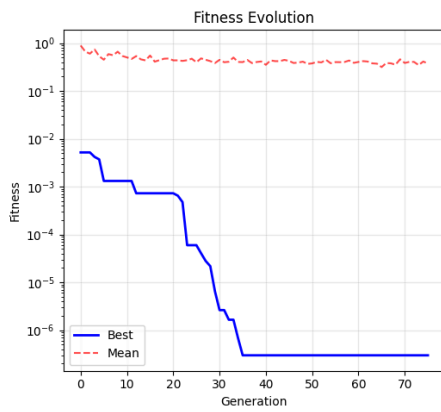
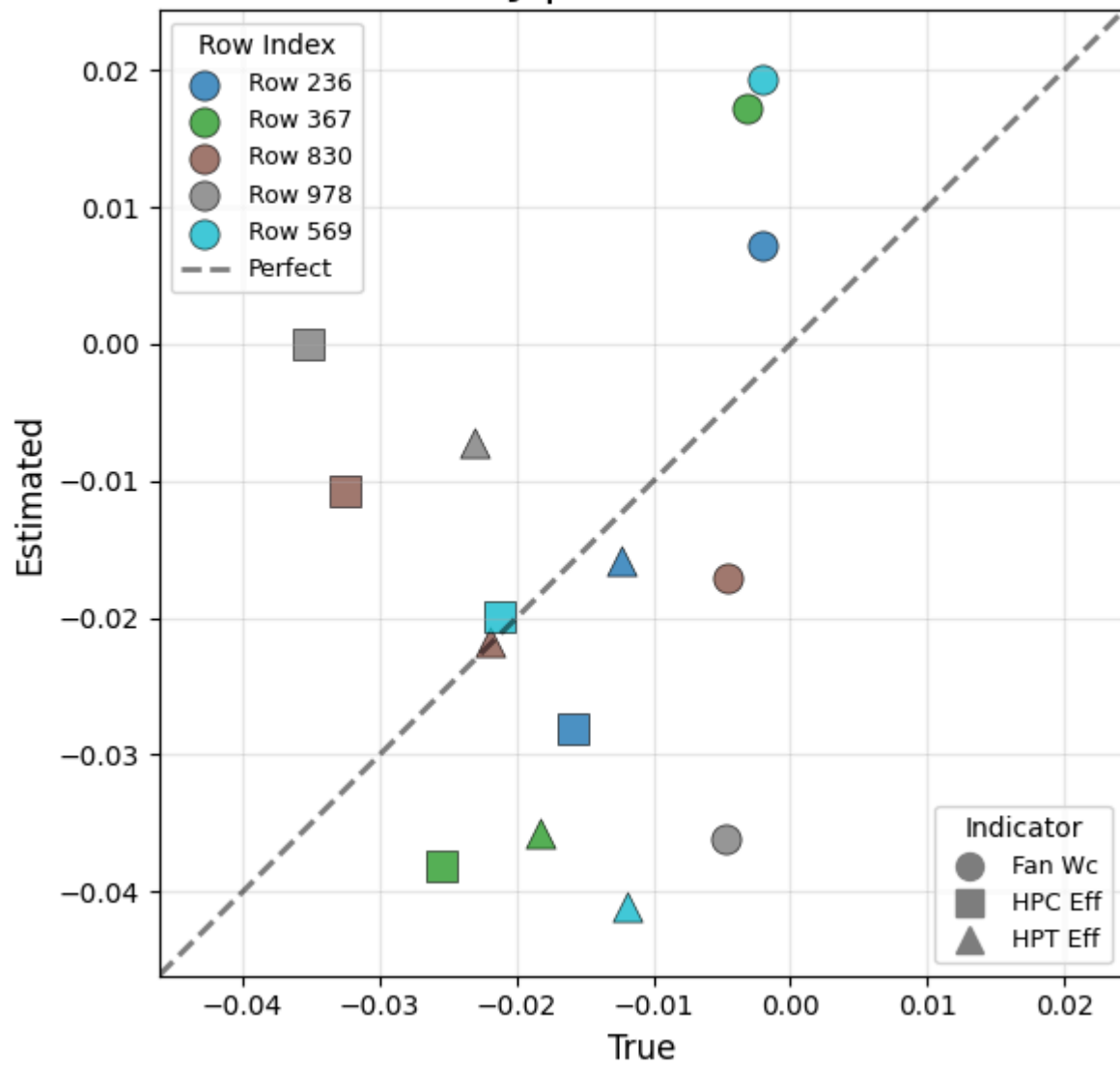
Running experiments on NOISY data

lctx_lsensors: ['CLIMB2'] × ['HPC_Tin']
1 measurements → 3 indicators

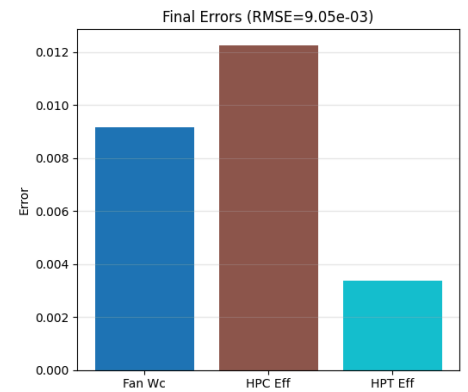
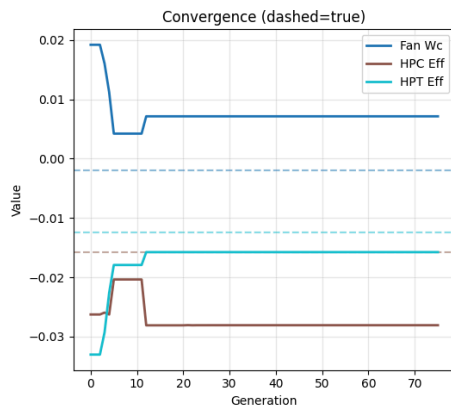
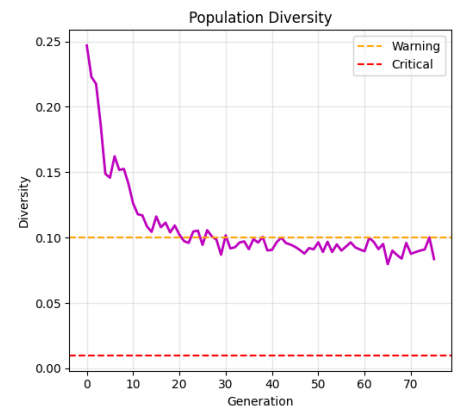
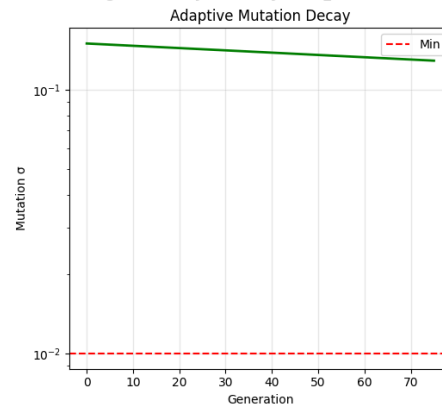
Noisy | 1 ctx × 1 sens



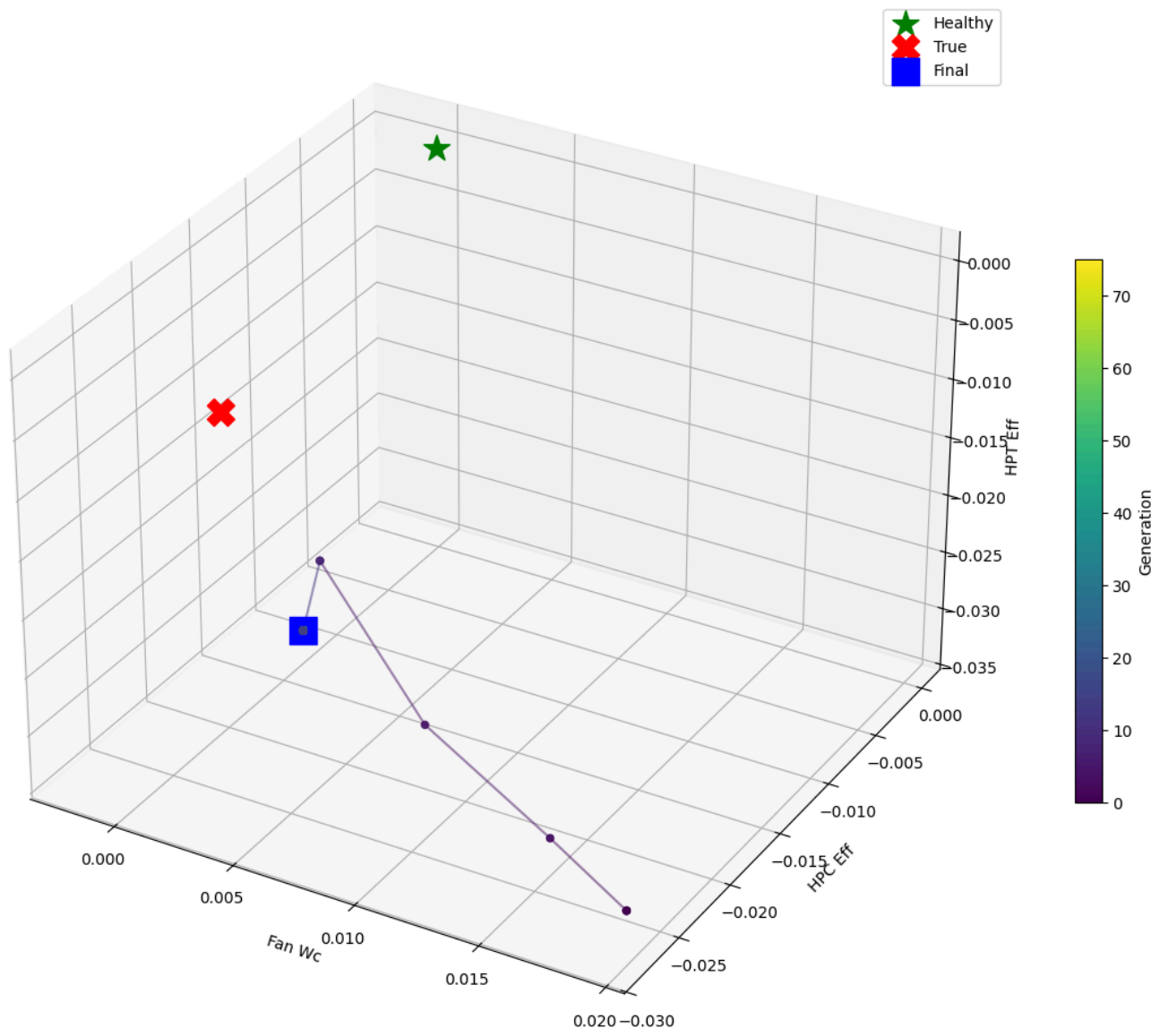
Noisy | 1 ctx × 1 sens



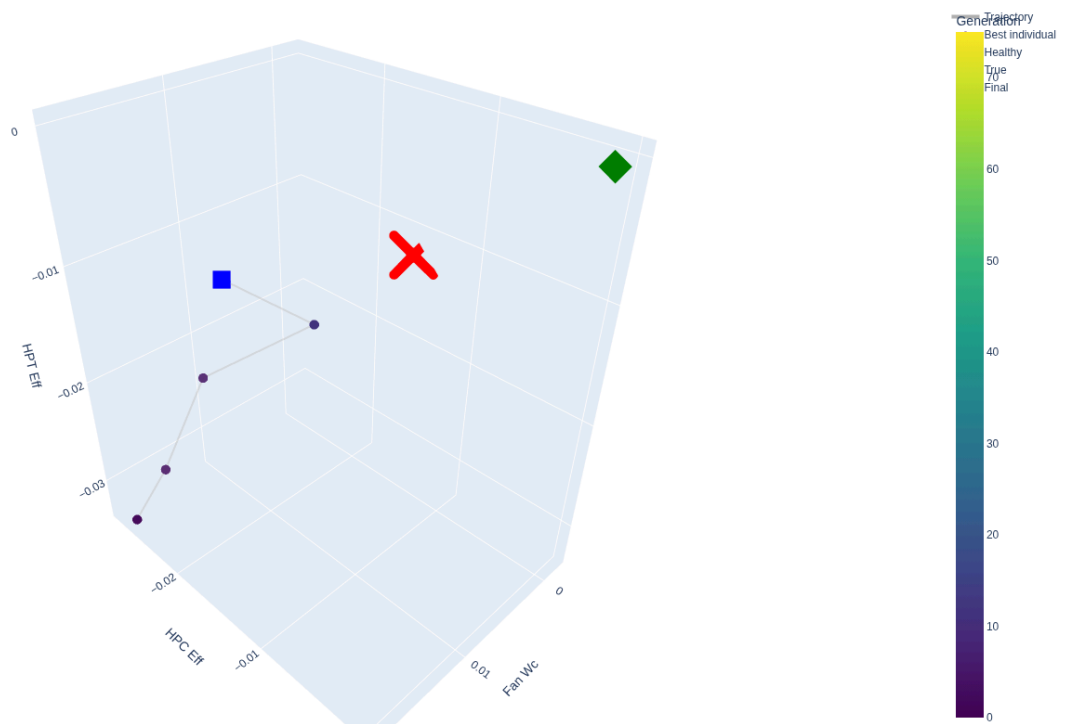
Convergence Analysis (noisy, 1ctx_1sensors)



3D Search Trajectory (noisy, 1ctx_1sensors)



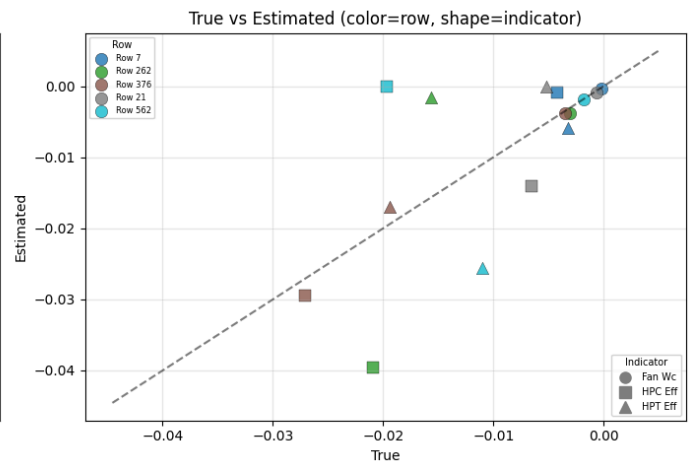
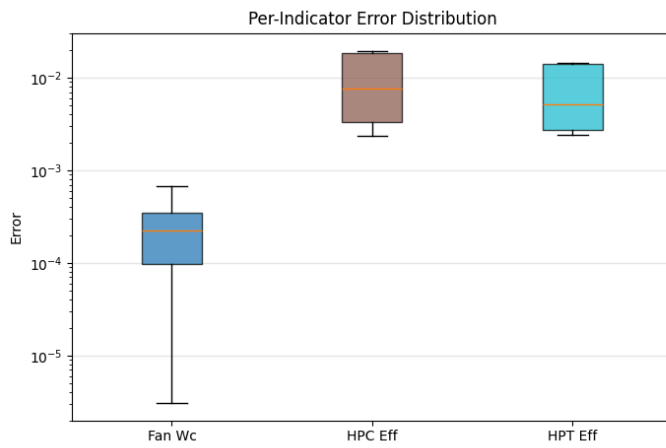
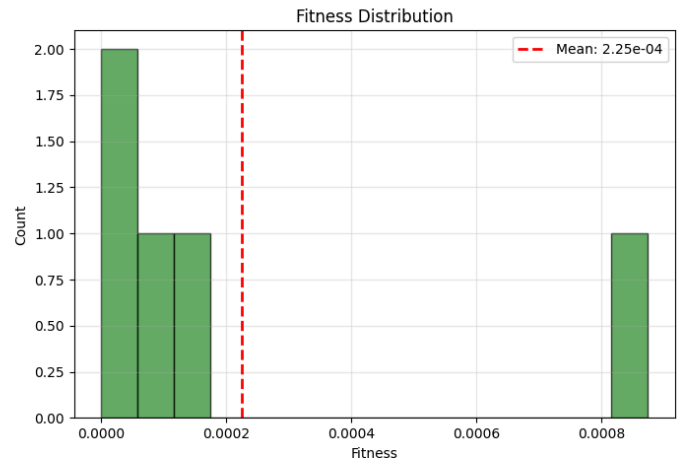
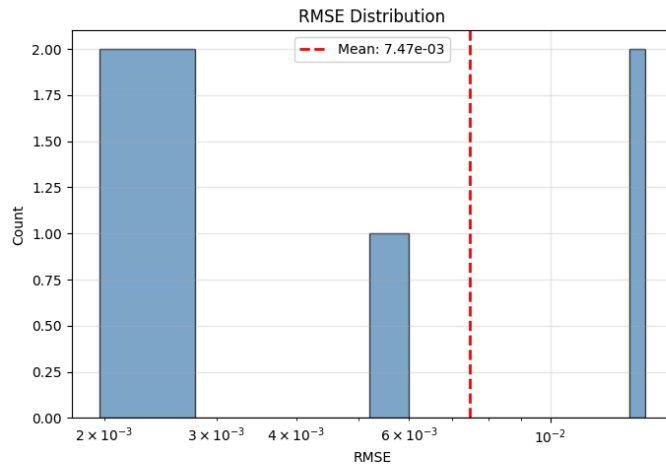
GA Search Trajectory



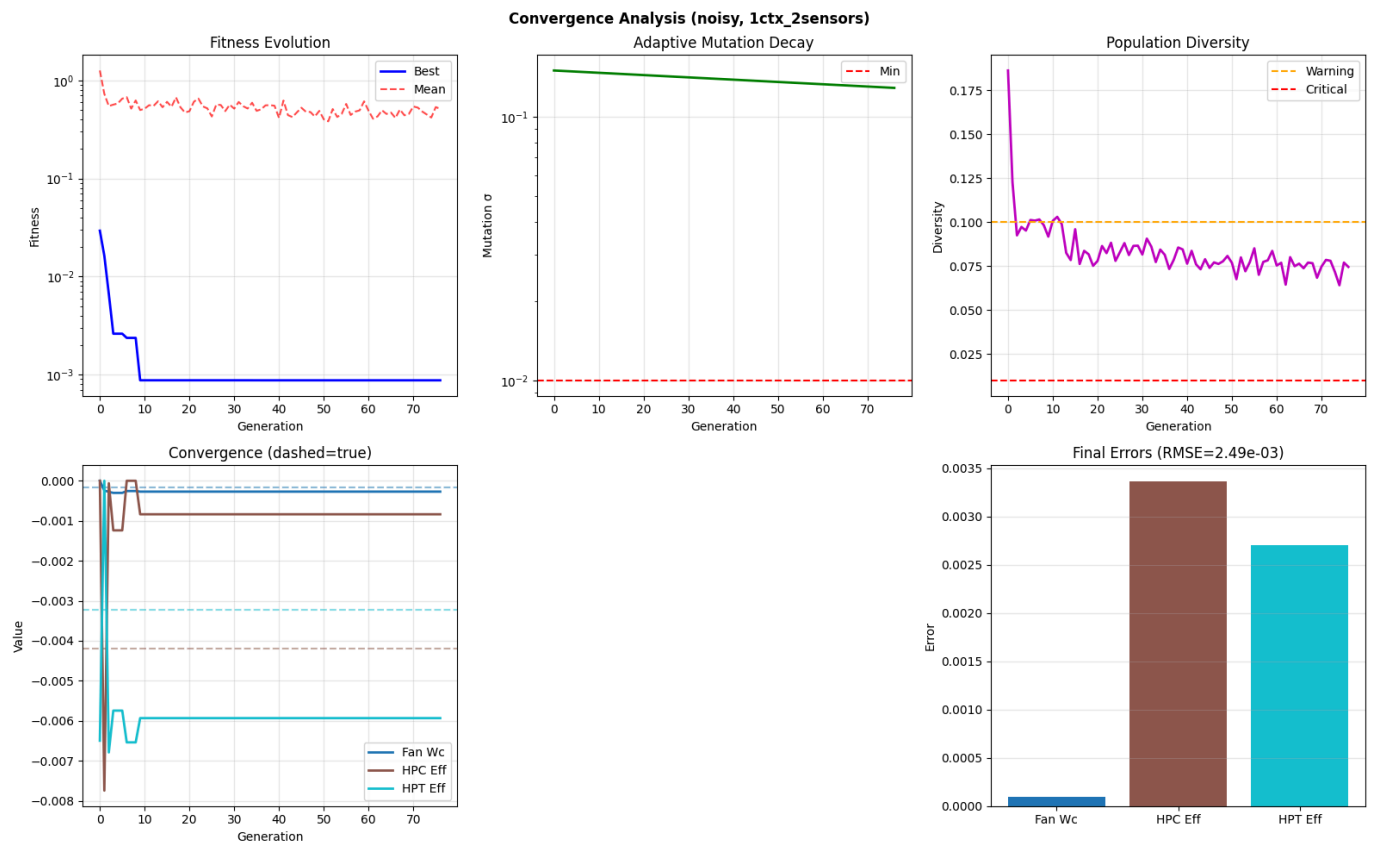
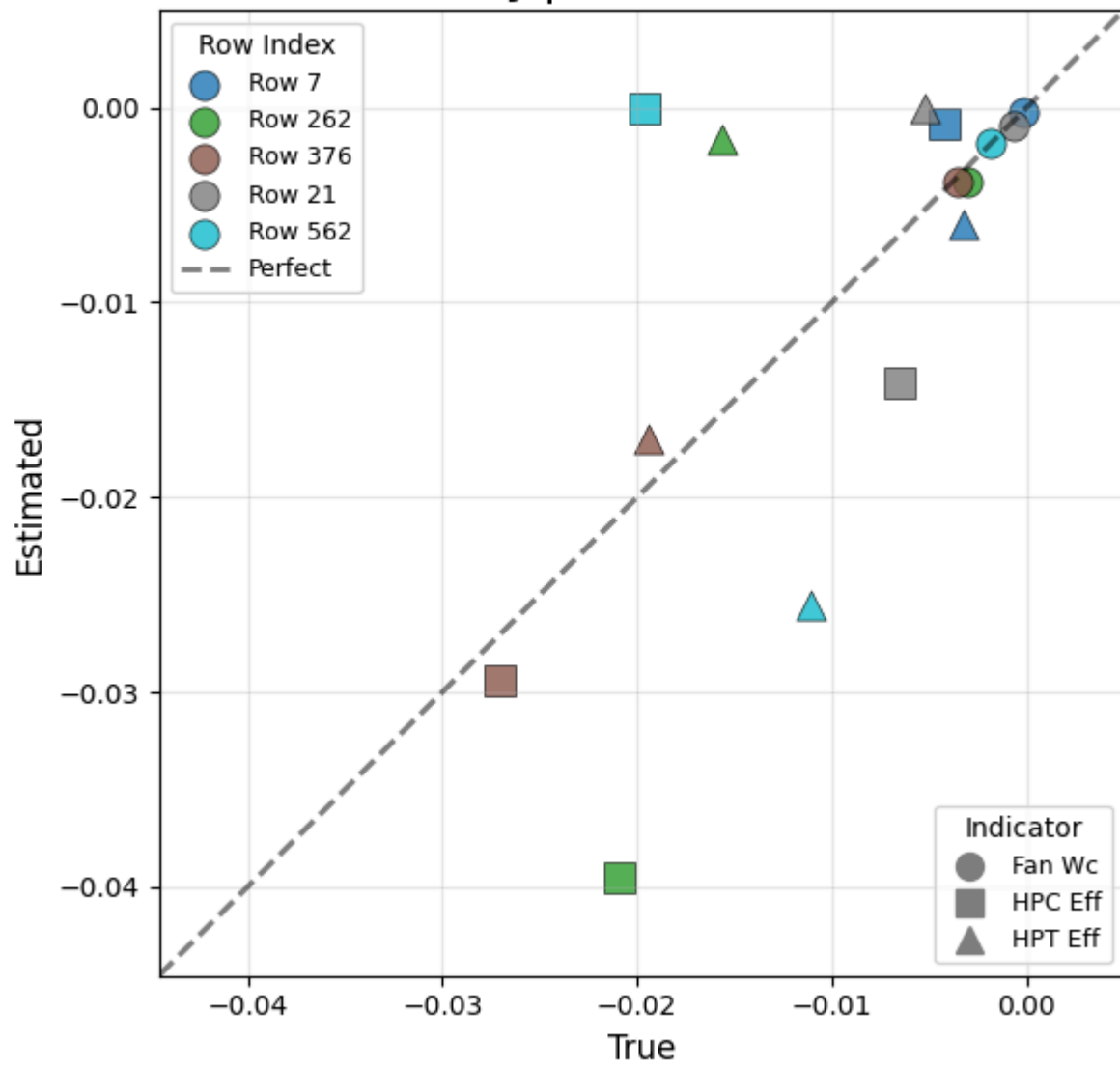
Mean RMSE: $1.81\text{e-}02 \pm 7.34\text{e-}03$
Mean Time: 1742.62s $\pm$ 372.40s

lctx_2sensors: ['CLIMB2'] $\times$ ['HPC_Tin', 'LPT_Tin']
2 measurements $\rightarrow$ 3 indicators

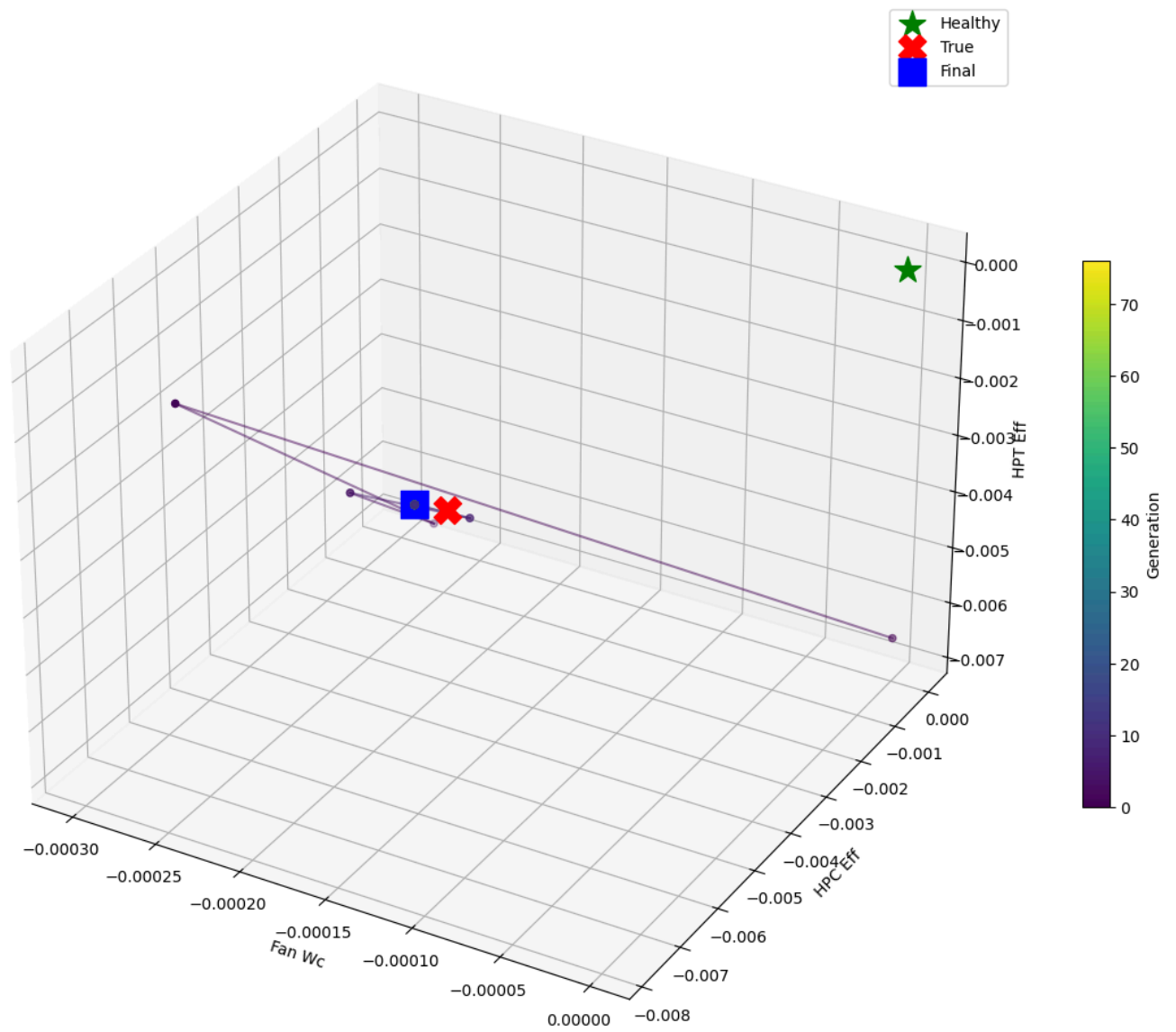
Noisy | 1 ctx $\times$ 2 sens



Noisy | 1 ctx × 2 sens



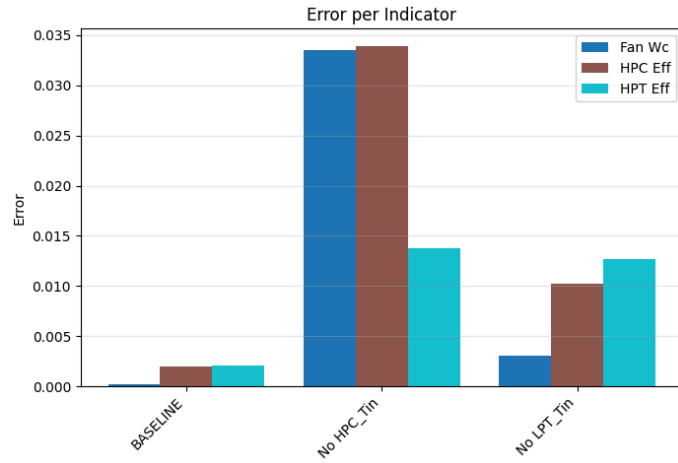
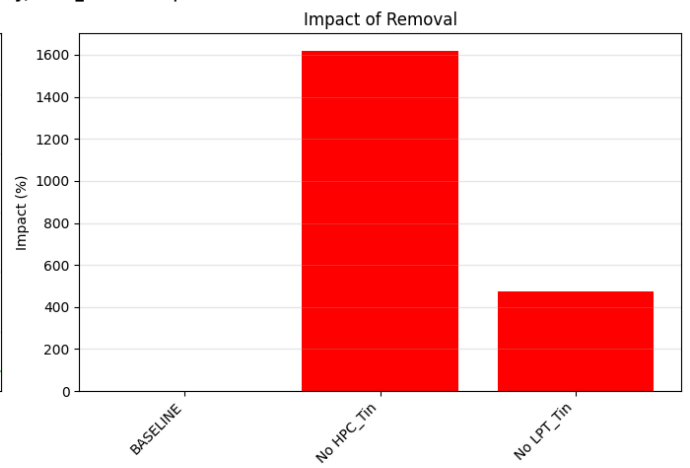
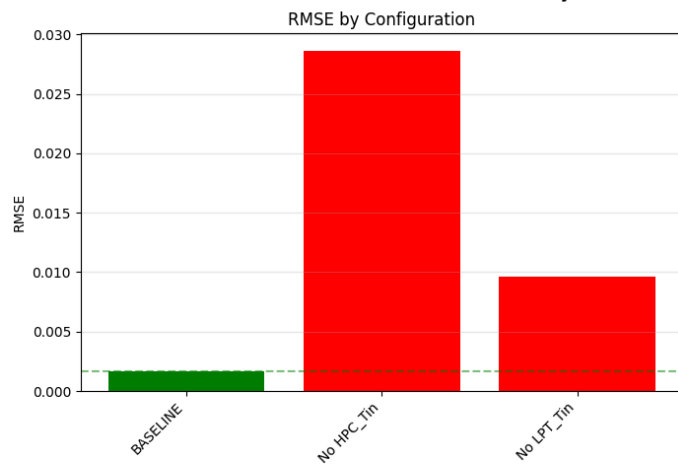
3D Search Trajectory (noisy, 1ctx_2sensors)



Running ablation study...
Running BASELINE...
RMSE: 1.67e-03

Sensor ablation:
Without HPC_Tin... RMSE: 2.86e-02 (+1620.6%)
Without LPT_Tin... RMSE: 9.59e-03 (+476.0%)

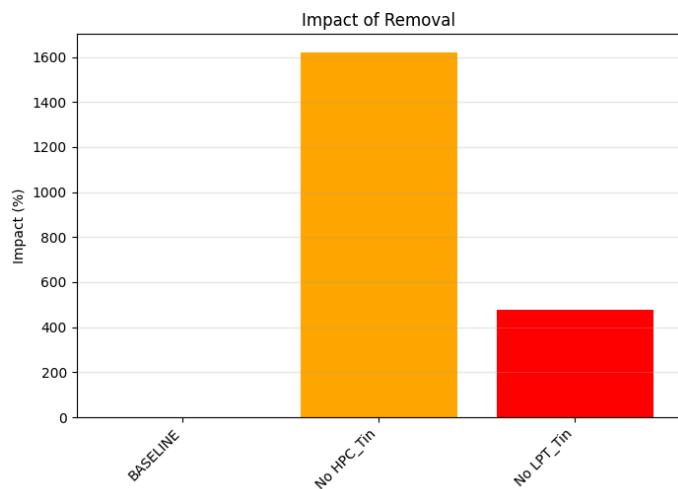
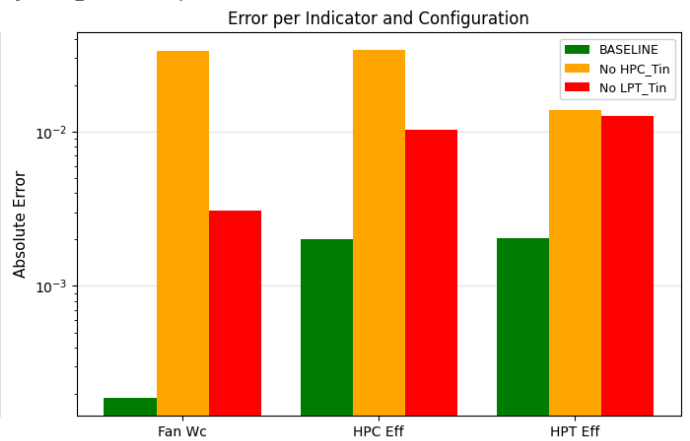
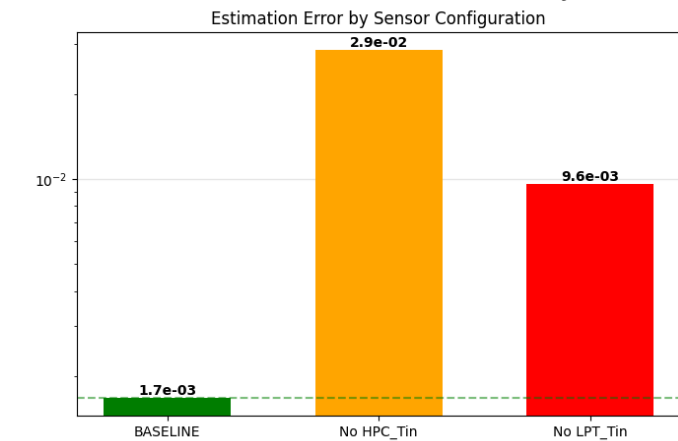
Ablation Study Results (noisy, 1ctx_2sensors | row 53)



INTERPRETATION:
 BASELINE: RMSE = 1.67e-03
 HPC_Tin: HIGH IMPORTANCE (+1620.6%)
 LPT_Tin: HIGH IMPORTANCE (+476.0%)

Legend:
 DETRIMENTAL: Removing HELPS
 NEUTRAL: Little effect
 HIGH IMPORTANCE: Removing HURTS

Ablation Study Results (noisy, 1ctx_2sensors | row 53)



INTERPRETATION:
 BASELINE: RMSE = 1.67e-03
 HPC_Tin: HIGH IMPORTANCE (+1620.6%)
 LPT_Tin: HIGH IMPORTANCE (+476.0%)

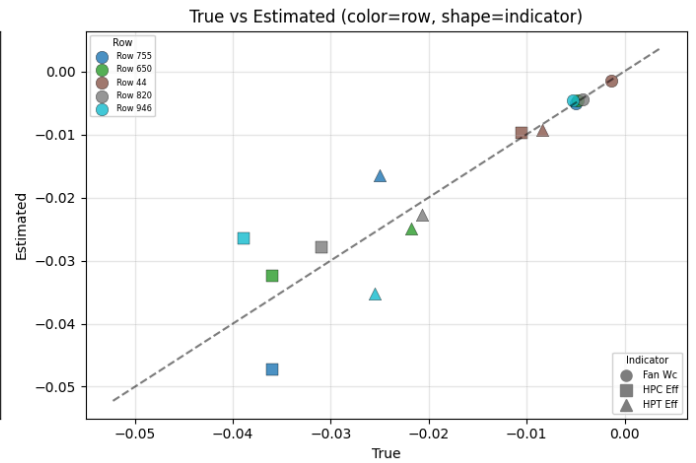
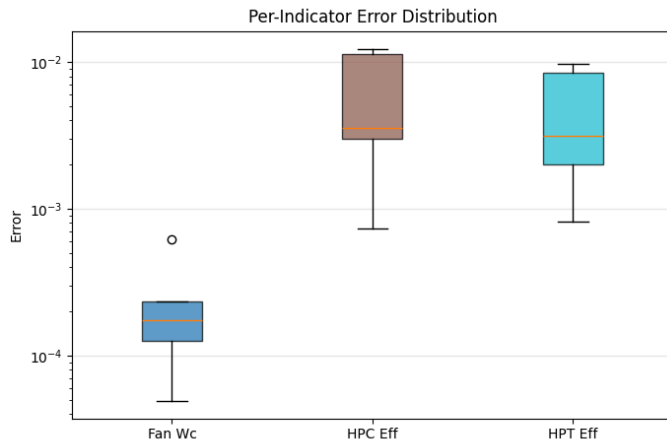
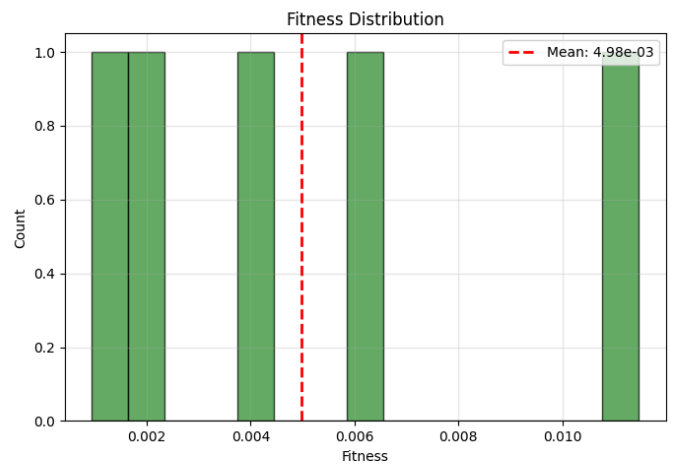
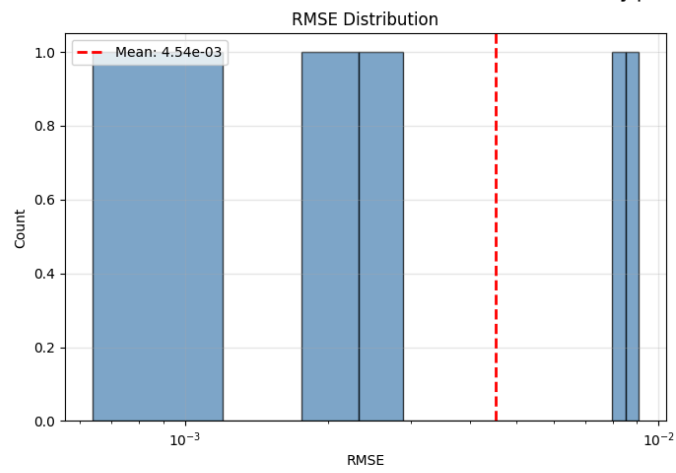
Legend:
 DETRIMENTAL: Removing HELPS
 NEUTRAL: Little effect
 HIGH IMPORTANCE: Removing HURTS

Mean RMSE: 7.47e-03 ± 5.91e-03

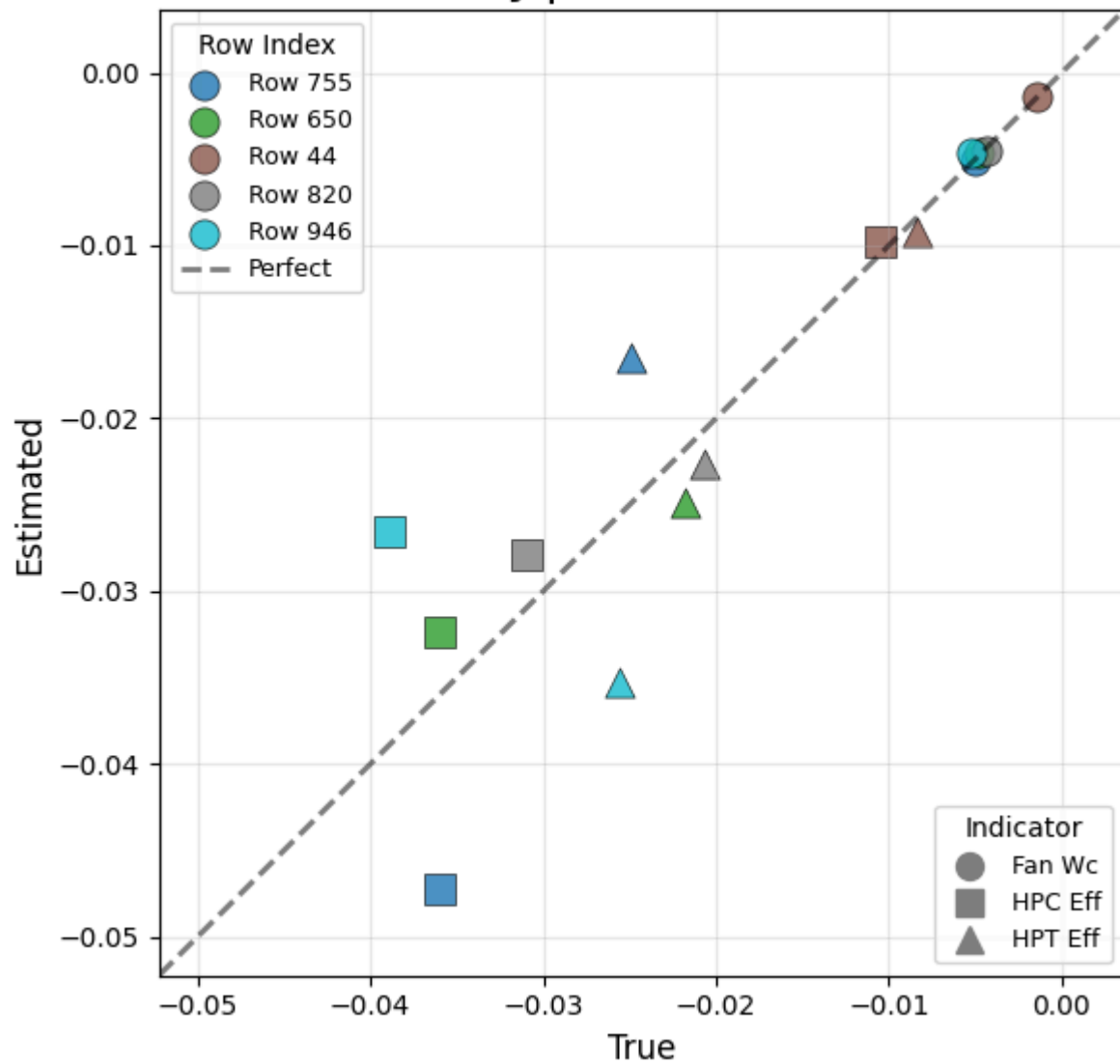
Mean Time: 2053.03s ± 270.94s

1ctx_3sensors: ['CLIMB2'] × ['HPC_Tin', 'LPT_Tin', 'HPC_Pout_st']
 3 measurements → 3 indicators

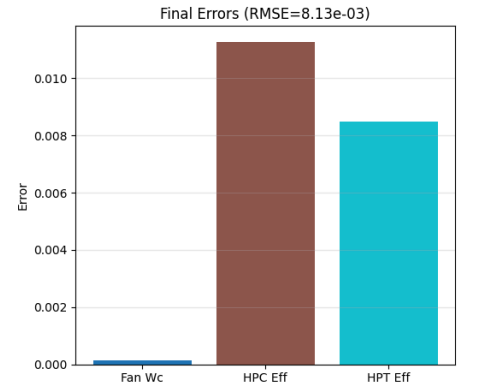
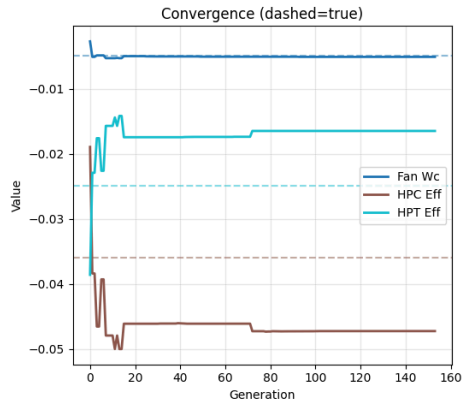
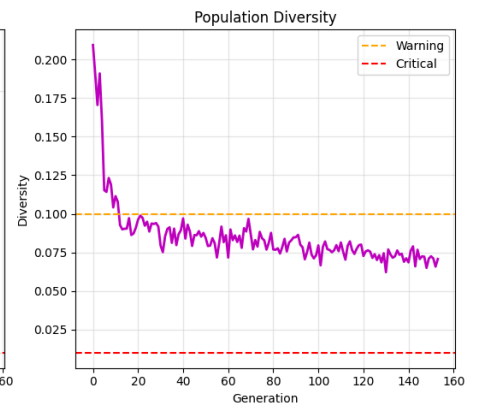
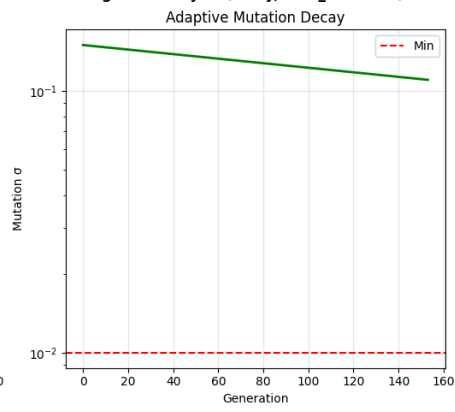
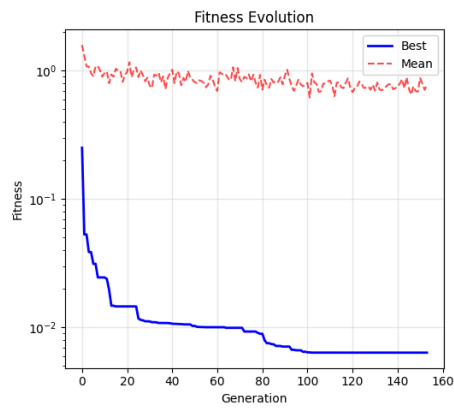
Noisy | 1 ctx x 3 sens



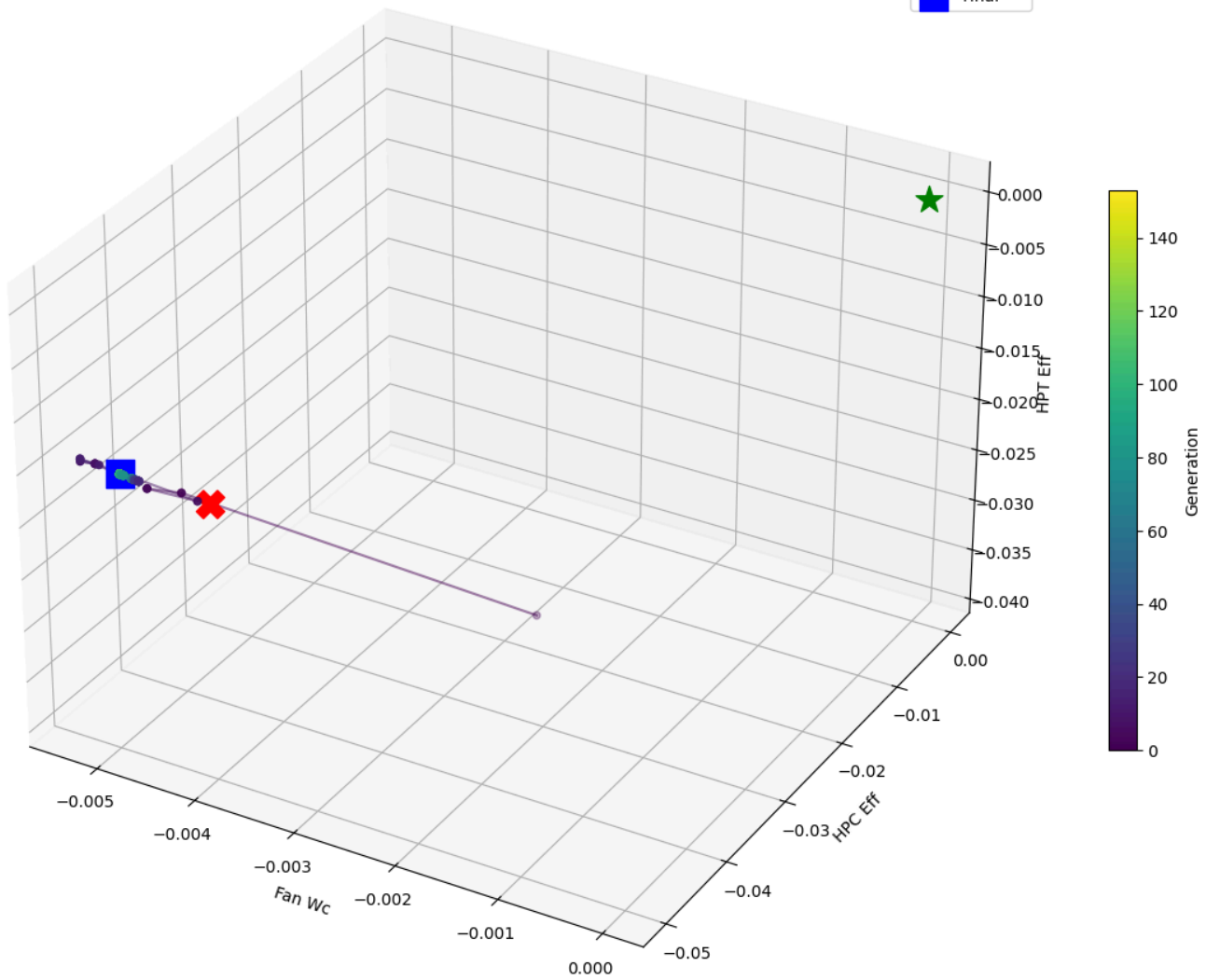
Noisy | 1 ctx x 3 sens



Convergence Analysis (noisy, 1ctx_3sensors)



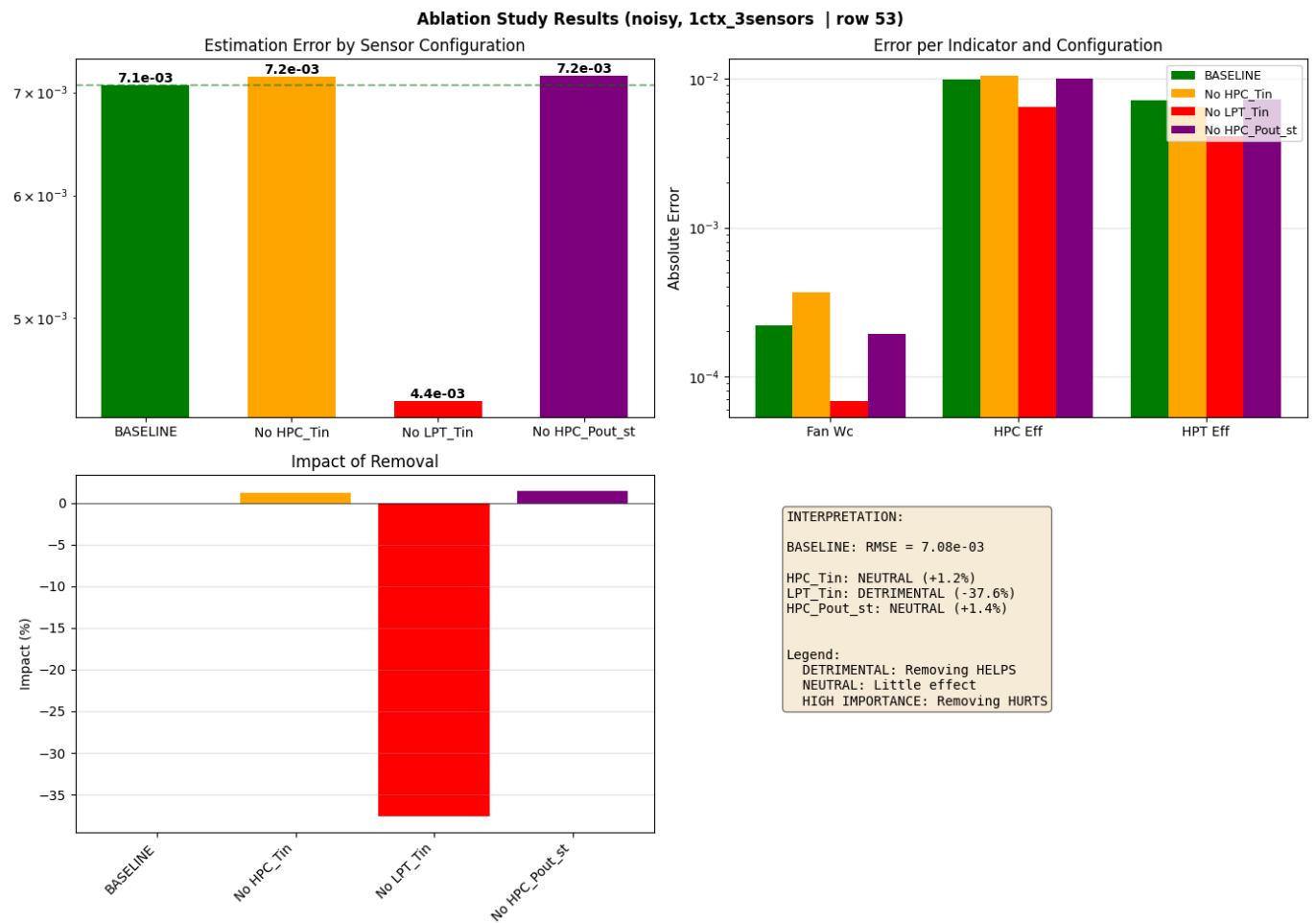
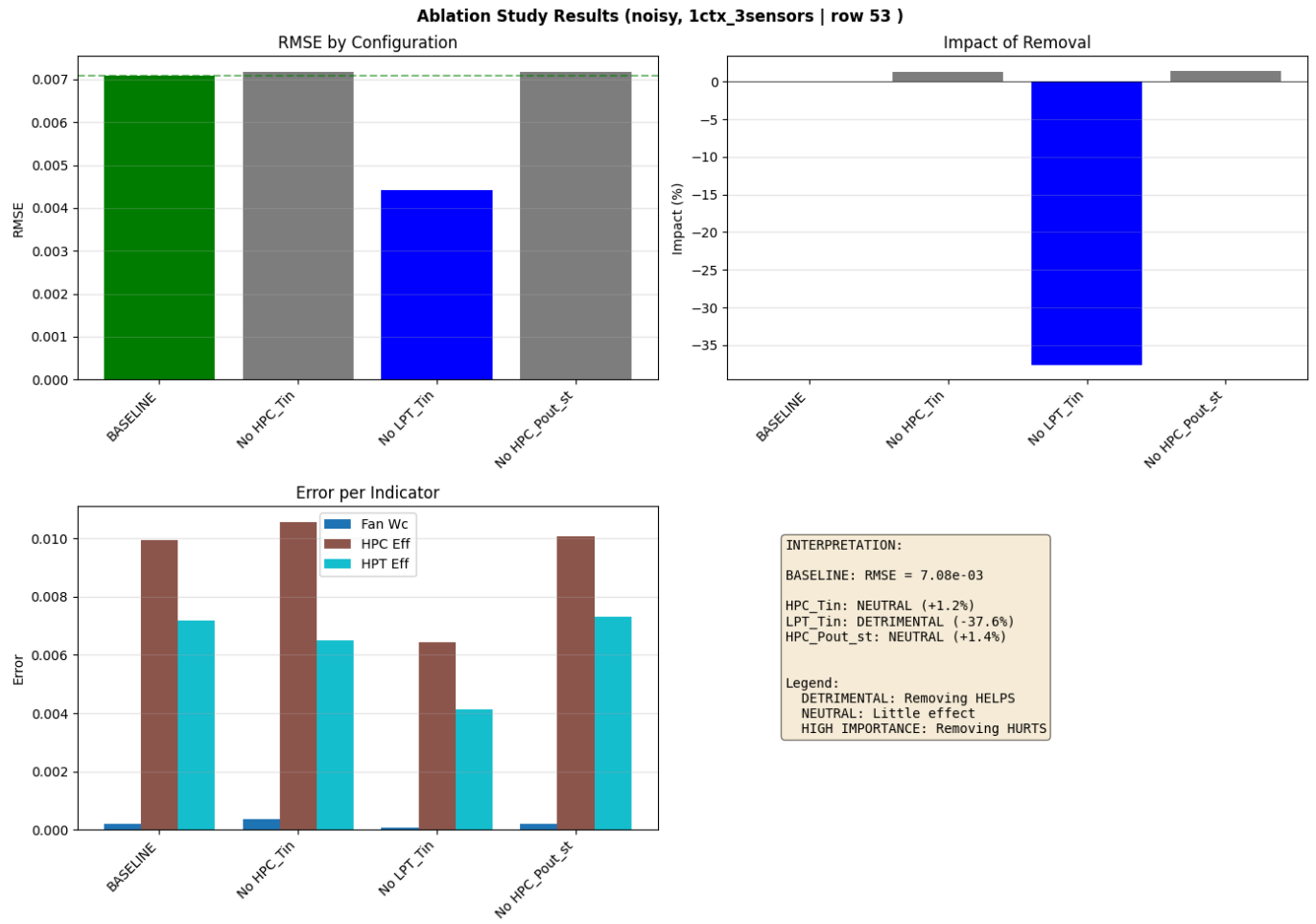
3D Search Trajectory (noisy, 1ctx_3sensors)



Running ablation study...
Running BASELINE...
RMSE: 7.08e-03

Sensor ablation:

Without HPC_Tin... RMSE: 7.17e-03 (+1.2%)
Without LPT_Tin... RMSE: 4.42e-03 (-37.6%)
Without HPC_Pout_st... RMSE: 7.18e-03 (+1.4%)



Mean RMSE: $4.54e-03 \pm 3.81e-03$
Mean Time: 5012.46s $\pm$ 926.95s
CPU times: user 1h 4min 31s, sys: 1min 43s, total: 1h 6min 14s
Wall time: 1d 12h 4min 12s

In []:

In []:

In []:

In []:

In []:

In []:

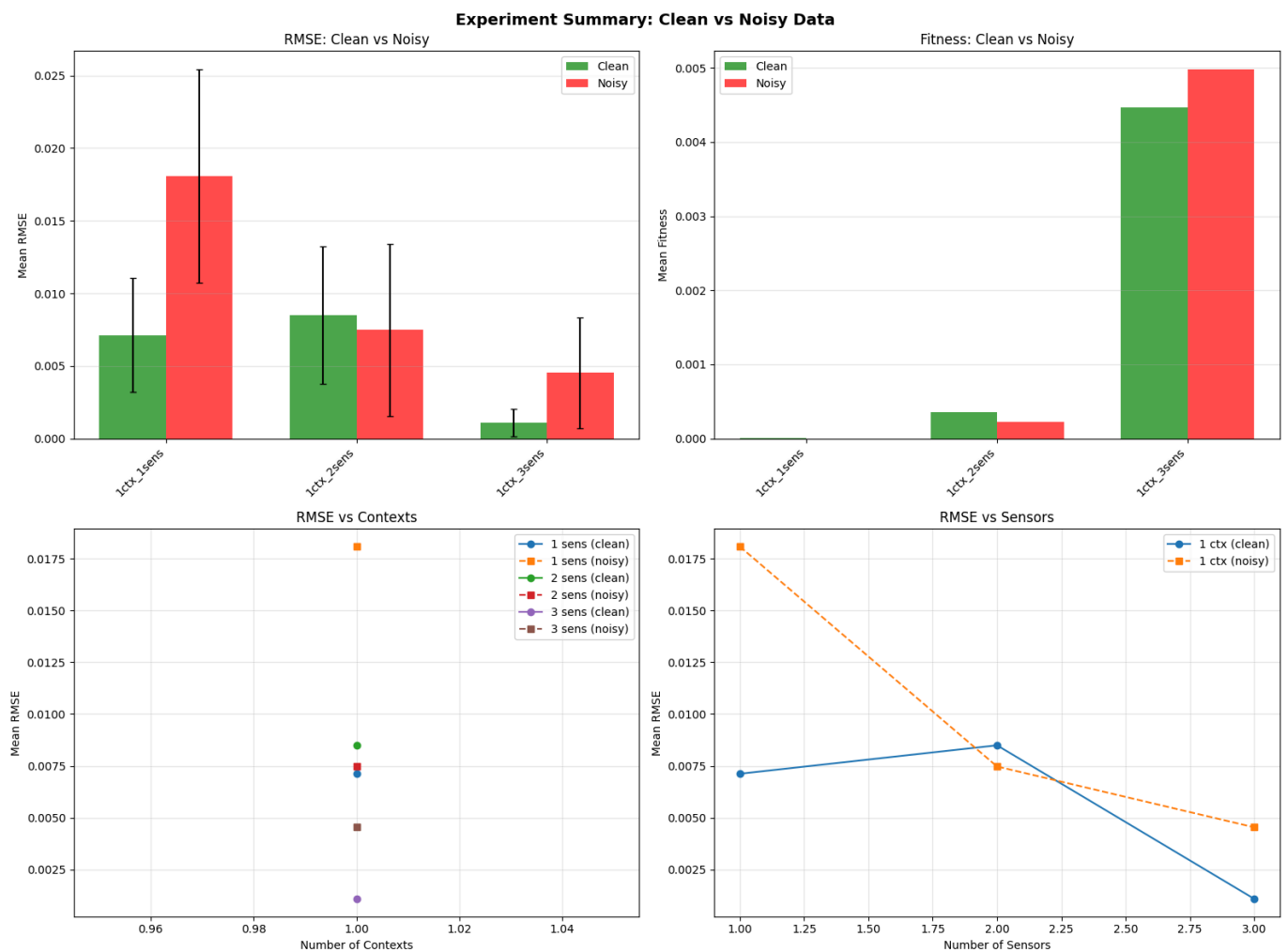
In []:

In []:

In []:

4. Summary Comparison Plot

```
In [20]: %%time
# Generate summary plot
plot_summary_comparison(summary_df, save_path=os.path.join(output_dir, "summary_compa
```



CPU times: user 719 ms, sys: 30 ms, total: 749 ms
Wall time: 747 ms

In []:

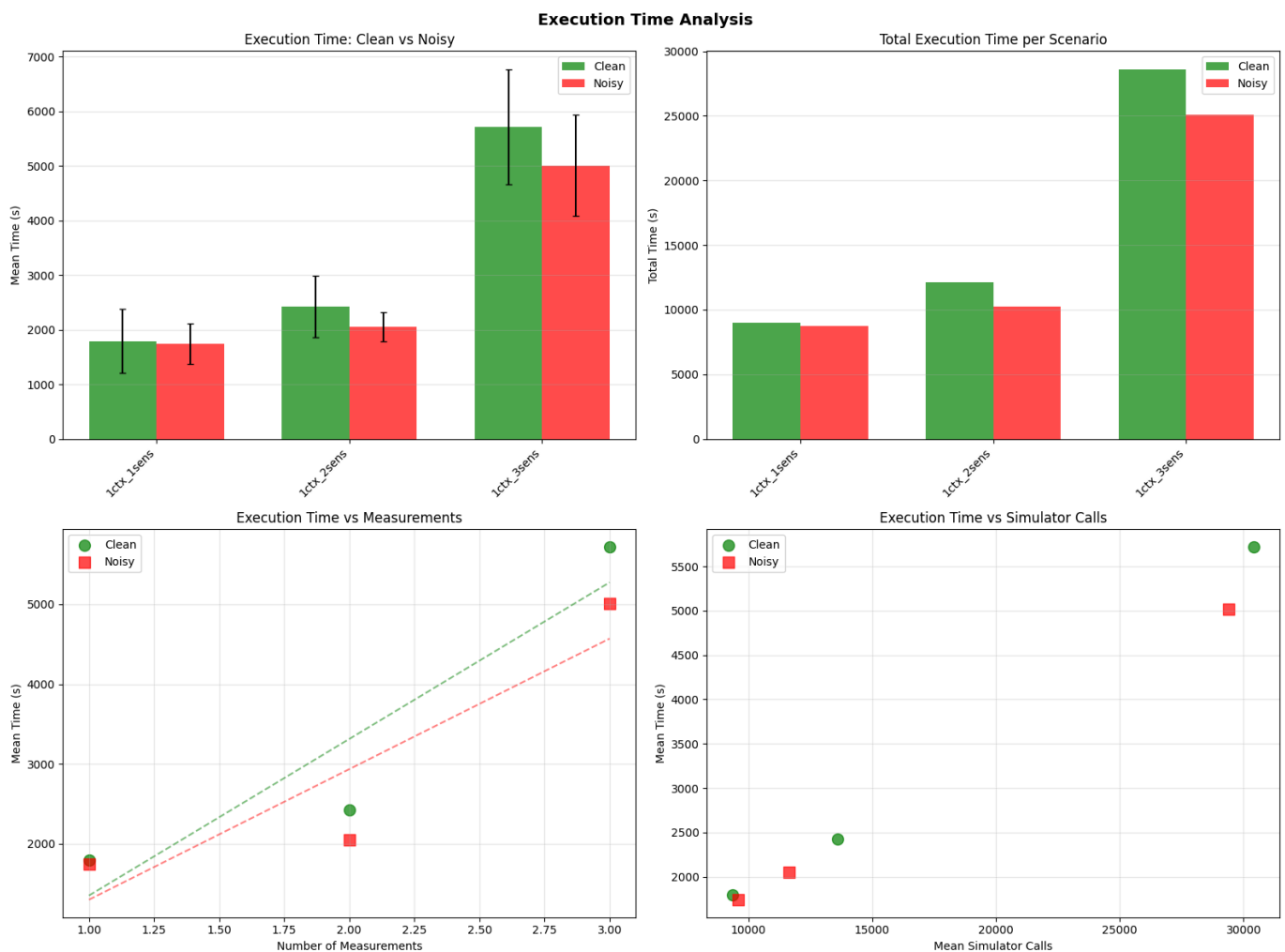
In []:

In []:

In []:

5. PLOT execution time

```
In [21]: %%time
# Generate execution time plot
plot_execution_times(summary_df, save_path=os.path.join(output_dir, "execution_time_a
print(f"Execution time plot saved to: {os.path.join(output_dir, 'execution_time_analy
```



Execution time plot saved to: ../experiments/run_20260127_060413/execution_time_analysis.png

CPU times: user 696 ms, sys: 21 ms, total: 717 ms
Wall time: 716 ms

6. Summary of the experiemnt

Summary of the results over multiple index row

```
In [22]: # Display summary table
print("\n" + "="*100)
print("EXPERIMENT SUMMARY")
print("="*100)
print(summary_df[['data_type', 'n_contexts', 'n_sensors', 'n_measurements',
```

```

        'mean_rmse', 'std_rmse', 'mean_fitness', 'mean_time', 'std_time',
        'mean_sim_calls', 'mean_generations']]).to_string(index=False))

# Print timing summary
print("\n" + "="*100)
print("TIMING SUMMARY")
print("="*100)
clean_total = summary_df[summary_df['data_type']=='clean']['total_time'].sum()
noisy_total = summary_df[summary_df['data_type']=='noisy']['total_time'].sum()
print(f"Total time for CLEAN experiments: {clean_total:.1f}s ({clean_total/60:.1f} mi
print(f"Total time for NOISY experiments: {noisy_total:.1f}s ({noisy_total/60:.1f} mi
print(f"Grand total: {clean_total + noisy_total:.1f}s ({(clean_total + noisy_total)/6

```

===== ===== EXPERIMENT SUMMARY ===== =====

data_type	n_contexts	n_sensors	n_measurements	mean_rmse	std_rmse	mean_fitness
mean_time	std_time	mean_sim_calls	mean_generations			
clean	1	1	1	0.007120	0.003920	4.685851e-06 17
95.273549	579.435271	9377.6	69.0			
clean	1	2	2	0.008493	0.004748	3.609711e-04 24
30.189519	560.750072	13598.0	100.8			
clean	1	3	3	0.001088	0.000945	4.471539e-03 57
17.001706	1050.805014	30401.0	221.8			
noisy	1	1	1	0.018079	0.007340	3.033411e-07 17
42.624219	372.404173	9584.6	72.0			
noisy	1	2	2	0.007472	0.005915	2.251944e-04 20
53.033018	270.943200	11635.8	88.2			
noisy	1	3	3	0.004537	0.003809	4.978638e-03 50
12.464643	926.954707	29409.6	215.2			

===== ===== TIMING SUMMARY ===== =====

Total time for CLEAN experiments: 49712.3s (828.5 min)

Total time for NOISY experiments: 44040.6s (734.0 min)

Grand total: 93752.9s (1562.5 min)

```

In [23]: # Print output directory structure
print(f"\n\nAll results saved in: {output_dir}")
print("\nDirectory structure:")
for root, dirs, files in os.walk(output_dir):
    level = root.replace(output_dir, '').count(os.sep)
    indent = ' ' * 2 * level
    print(f"{indent}{os.path.basename(root)}/")
    subindent = ' ' * 2 * (level + 1)
    for file in files[:5]: # Limit files shown
        print(f"{subindent}{file}")
    if len(files) > 5:
        print(f"{subindent}... and {len(files)-5} more files")

```

All results saved in: ../experiments/run_20260127_060413

Directory structure:

```
run_20260127_060413/  
  execution_time_analysis.png  
  summary_comparison.png  
  experiment_summary.csv  
  clean/  
    lctx_1sensors/  
      trajectory_3d.png  
      multiple_solutions.csv  
      convergence.png  
      results_summary.png  
      trajectory_3d.html  
      ... and 2 more files  
    lctx_3sensors/  
      trajectory_3d.png  
      ablation.csv  
      multiple_solutions.csv  
      convergence.png  
      results_summary.png  
      ... and 5 more files  
    lctx_2sensors/  
      trajectory_3d.png  
      ablation.csv  
      multiple_solutions.csv  
      convergence.png  
      results_summary.png  
      ... and 5 more files  
  noisy/  
    lctx_1sensors/  
      trajectory_3d.png  
      multiple_solutions.csv  
      convergence.png  
      results_summary.png  
      trajectory_3d.html  
      ... and 2 more files  
    lctx_3sensors/  
      trajectory_3d.png  
      ablation.csv  
      multiple_solutions.csv  
      convergence.png  
      results_summary.png  
      ... and 5 more files  
    lctx_2sensors/  
      trajectory_3d.png  
      ablation.csv  
      multiple_solutions.csv  
      convergence.png  
      results_summary.png  
      ... and 5 more files
```

In []: